



UNIVERSITÀ DEGLI STUDI DI BERGAMO

PAC – Progettazione Algoritmi e Computabilità

Sfrigo

Web App per la Gestione del Frigorifero Condiviso

Autori:

Paganelli Thomas

Matricola n. 1085805

Petrisor Andrei

Matricola n. 1085993

Radu Antonio Valentin

Matricola n. 1085992

Docente:

Prof.ssa Scandurra Patrizia

Anno Accademico 2025/2026

Indice

1	Contesto e Motivazioni	4
1.1	Contesto Applicativo	4
1.2	Il Problema che Sfrigo Risolve	4
1.3	Obiettivi del Progetto	5
2	Componenti del Progetto	6
2.1	Struttura a Macroblocchi	6
2.1.1	Frontend – Next.js Application	6
2.1.2	API Gateway	7
2.1.3	User Service	7
2.1.4	Fridge Service	7
2.1.5	Recipe Service	7
3	Architettura e Metodologia	9
3.1	Architettura a Microservizi	9
3.2	Scelta dei Database: SQL e NoSQL	10
3.2.1	PostgreSQL – Dati Relazionali	10
3.2.2	MongoDB – Dati Eterogenei e Flessibili	11
3.3	Toolchain e Motivazioni	11
3.4	Versionamento e Gestione dei Microservizi	13
4	Metodologia di Sviluppo	14
4.1	Approccio Agile e Framework Scrum	14
4.2	Agile Model Driven Development (AMDD)	14
4.3	DevOps e Integrazione Continua	15
5	Analisi del Codice	16
5.1	Analisi Statica	16
5.2	Analisi Dinamica	16
6	Iterazione 0	18

6.1	Analisi dei Requisiti	18
6.1.1	Gestione Account e Accesso	18
6.1.2	Gestione Frigorifero	18
6.1.3	Gestione Membri e Inviti	18
6.1.4	Gestione Inventario	19
6.1.5	Supporto alla Creazione di Ricette	19
6.2	Use Case Diagram	19
6.3	Analisi delle Specifiche e Prioritizzazione	20
6.3.1	Alta Priorità	20
6.3.2	Media Priorità	20
6.3.3	Bassa Priorità	21
6.4	Analisi dell'Architettura – Deployment Diagram	21
7	Iterazione 1	23
7.1	Introduzione	23
7.2	Descrizione dei Casi d'Uso	24
7.2.1	UC1 – Visualizzazione Schermata Iniziale	24
7.2.2	UC1.1 – Registrazione Utente	25
7.2.3	UC1.2 – Login Utente	26
7.2.4	UC1.3 – Logout Utente	27
7.2.5	UC2 – Visualizzazione Dashboard	28
7.2.6	UC2.1 – Creazione Frigorifero	29
7.2.7	UC3 – Visualizzazione Inventario	30
7.2.8	UC3.1 – Aggiungi Alimento	30
7.2.9	UC3.2 – Rimozione Alimento	31
7.2.10	UC3.3 – Invito al Frigo	32
7.2.11	UC3.4 – Rimozione Frigorifero	33
7.3	Component Diagram	34
7.4	UML Sequence Diagram	34
7.4.1	Autenticazione	35
7.4.2	Aggiunta di un Alimento	36
7.4.3	Creazione Frigorifero	37
7.4.4	Invito Utenti al Frigorifero	38
7.4.5	Gestione Frigorifero (Cancellazione)	39
7.4.6	Visualizzazione Alimenti del Frigorifero	40
7.4.7	Visualizzazione Lista Frigoriferi	41
7.5	UML Class Diagram – Schema dei Dati	42
7.6	Testing	43

7.6.1	Analisi Statica – ESLint	43
7.6.2	Analisi Dinamica – Jest e Supertest	43
7.7	Documentazione API	46
7.7.1	Add Item e Delete Item	46
7.7.2	Get Frigos e Get Items	47
7.7.3	Login tramite Firebase	48
8	Iterazione 2	49
8.1	Introduzione	49
8.2	Descrizione dei Casi d’Uso	50
8.2.1	UC5 – Alimento di Uso Comune	50
8.2.2	UC6 – Tracciamento Proprietario Alimento	51
8.2.3	UC7 – Richiesta Ricetta Personale	52
8.2.4	UC8 – Richiesta Ricetta Condivisa	53
8.2.5	UC9 – Algoritmo di Sorting	54
8.2.6	UC10 – Algoritmo di Equità	56
8.2.7	UC11 – Uscita Volontaria dal Frigorifero	59
8.2.8	UC12 – Rimozione Utente dal Frigorifero	59
8.3	Sequence Diagram – Iterazione 2	60
8.3.1	Generazione Ricetta	60
8.3.2	Calcolo Equità	61
8.3.3	Rimozione Utente dal Frigorifero	62
8.4	Testing – Iterazione 2	63
8.4.1	Analisi Dinamica – Fridge Service e Recipe Service	63
8.5	Documentazione API – Iterazione 2	64
8.5.1	Calcolo Equità e Generazione Ricetta	64
8.5.2	Rimozione Membro	64
9	Infrastruttura CI/CD	66
9.1	Continuous Integration e Continuous Delivery	66
9.2	Triggers e Filtri di Percorso	66
9.3	Flusso della Pipeline	67
9.4	Architettura di Esecuzione	69
9.5	Architettura di Deployment e Topologia dei Servizi	70
9.6	Implicazioni Strategiche e Aziendali	71
10	Viste Frontend	72

1. Contesto e Motivazioni

1.1 Contesto Applicativo

La condivisione di spazi abitativi è una realtà sempre più diffusa, in particolare tra studenti universitari, giovani lavoratori e famiglie allargate. In questi contesti, la gestione delle risorse comuni, e in primo luogo degli alimenti conservati in frigorifero, rappresenta una fonte ricorrente di inefficienze e attriti: alimenti scaduti non vengono rimossi tempestivamente, gli acquisti si duplicano senza necessità, la proprietà dei singoli prodotti è spesso incerta e, in assenza di strumenti adeguati, la comunicazione tra coinquilini è frammentata e informale.

Il problema è aggravato da una tendenza culturale e logistica verso il consumo individuale anche in contesti di convivenza: senza un sistema strutturato, ogni utente tende a gestire i propri alimenti in modo isolato, perdendo l'opportunità di valorizzare le risorse condivise e di ridurre gli sprechi.

1.2 Il Problema che Sfrigo Risolve

Sfrigo nasce per rispondere a una serie di problemi concreti e quotidiani:

- **Spreco alimentare:** in assenza di un sistema di monitoraggio, gli alimenti vengono dimenticati e consumati oltre la data di scadenza. Sfrigo traccia ogni prodotto e lo ordina per urgenza di consumo, riducendo gli sprechi.
- **Mancanza di trasparenza sulla proprietà:** in un frigorifero condiviso non è sempre chiaro quali alimenti appartengano a chi. Sfrigo associa ogni alimento al suo proprietario, garantendo chiarezza e prevenendo conflitti.
- **Difficoltà nella collaborazione culinaria:** cucinare insieme richiede di sapere cosa è disponibile e di chi sono gli ingredienti. Il modulo di generazione ricette di Sfrigo interroga un modello linguistico di grandi dimensioni (LLM) per proporre ricette basate sugli alimenti effettivamente presenti, sia personali che condivisi.

- **Gestione disomogenea dei prodotti condivisi:** alcuni alimenti (sale, olio, condimenti) sono di uso comune, mentre altri sono strettamente personali. Sfrigo consente di marcare esplicitamente la natura di ogni prodotto e di distribuire equamente il contributo di ciascun membro tramite un algoritmo di equità.
- **Assenza di coordinamento nelle abitazioni condivise:** Sfrigo è progettato per abitazioni con più inquilini, residenze universitarie, famiglie e uffici, fornendo un'unica interfaccia che centralizza la gestione del frigorifero e riduce la necessità di comunicazione informale.

1.3 Obiettivi del Progetto

L'obiettivo generale del progetto è la realizzazione di una web application che permetta a gruppi di utenti di gestire in modo collaborativo uno o più frigoriferi virtuali condivisi. Nel dettaglio, il sistema si propone di:

- Ridurre gli sprechi alimentari attraverso il tracciamento delle scadenze e la prioritizzazione dei consumi.
- Facilitare la condivisione degli alimenti tra più utenti con un sistema di proprietà e condivisione trasparente.
- Migliorare l'organizzazione del frigorifero tramite categorizzazione, ricerca e ordinamento dinamico degli alimenti.
- Supportare la creazione di ricette basate sugli ingredienti disponibili, sfruttando modelli AI generativi.
- Garantire equità nella gestione delle risorse condivise tramite un algoritmo di distribuzione dei costi.

2. Componenti del Progetto

2.1 Struttura a Macroblocchi

Il sistema Sfrigo è articolato in cinque componenti principali, ciascuno con responsabilità ben definite e confini chiari. Questa struttura riflette il pattern architetturale a microservizi adottato, descritto in dettaglio nel capitolo successivo.

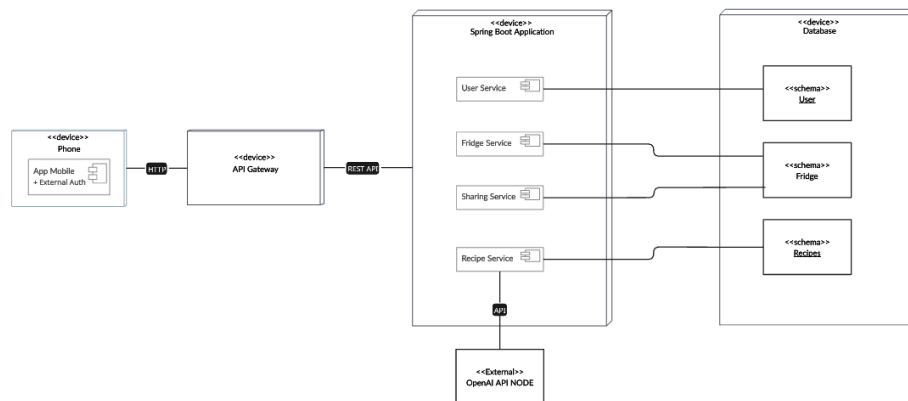


Figura 2.1: Visione d'insieme dell'architettura del sistema

2.1.1 Frontend – Next.js Application

L'interfaccia utente è realizzata come applicazione **Next.js** (v16), un framework React che supporta sia il rendering lato server (SSR) sia la generazione statica (SSG). Il frontend comunica esclusivamente con l'API Gateway tramite chiamate HTTP autenticate. Le pagine principali includono:

- / – Landing page pubblica
- /login, /register – Autenticazione e registrazione
- /dashboard – Pannello principale con elenco frigoriferi
- /dashboard/fridge/[id] – Vista inventario del singolo frigorifero
- /invite/[token] – Pagina di accettazione inviti

2.1.2 API Gateway

Il **Gateway** è il punto di ingresso unico per tutte le richieste provenienti dal frontend. Le sue responsabilità sono:

- Verificare i token JWT emessi da Firebase Authentication, rifiutando le richieste non autenticate.
- Iniettare le informazioni sull'utente autenticato (**X-User-Id**, **X-User-Email**) negli header delle richieste inoltrate ai microservizi downstream, evitando che ogni servizio debba gestire autonomamente l'autenticazione.
- Instradare le richieste al microservizio competente (User Service, Fridge Service, Recipe Service).

2.1.3 User Service

Il **User Service** gestisce il ciclo di vita del profilo utente all'interno del sistema. Sebbene l'autenticazione primaria sia delegata a Firebase, il servizio si occupa di creare e memorizzare il profilo applicativo dell'utente in PostgreSQL al momento della prima registrazione, associando l'UID Firebase a uno username univoco e a un indirizzo email.

2.1.4 Fridge Service

Il **Fridge Service** (porta 8082) è il microservizio di maggiore complessità e centralità nel sistema. Gestisce:

- La creazione, modifica e cancellazione dei frigoriferi virtuali (dati relazionali in PostgreSQL).
- La gestione dei membri e dei ruoli (OWNER/MEMBER) all'interno di ogni frigorifero.
- Il sistema di inviti tramite token temporanei con scadenza a 24 ore.
- L'inventario degli alimenti di ogni frigorifero (dati flessibili in MongoDB).
- Il calcolo dell'equità economica tra i membri basato sul valore degli alimenti inseriti.

2.1.5 Recipe Service

Il **Recipe Service** si integra con l'API OpenAI per generare suggerimenti di ricette a partire dagli alimenti disponibili nell'inventario. Riceve una lista di

ingredienti selezionati dall'utente, costruisce un prompt strutturato e restituisce le ricette generate dal modello GPT-4o-mini. Il servizio è stateless e non persiste dati propri.

3. Architettura e Metodologia

3.1 Architettura a Microservizi

L'architettura adottata è basata sul pattern dei **microservizi**, in cui il sistema è decomposto in un insieme di servizi indipendenti, ciascuno deployabile e scalabile autonomamente. Ogni microservizio espone le proprie funzionalità tramite **API REST** e comunica con gli altri esclusivamente attraverso lo strato di rete, senza condivisione diretta di memoria o stato.

Questa scelta architetturale è motivata da una serie di vantaggi concreti che si riflettono sullo sviluppo collaborativo del progetto:

Sviluppo parallelo e indipendente

La suddivisione in servizi con confini netti ha consentito ai membri del team di lavorare su componenti differenti senza interferenze reciproche. Ogni servizio ha il proprio `package.json`, il proprio ciclo di build e il proprio container Docker, riducendo al minimo le dipendenze trasversali.

Scalabilità selettiva

In un sistema monolitico, è necessario scalare l'intera applicazione anche se il collo di bottiglia riguarda un solo modulo. Con i microservizi, è possibile aumentare le repliche del solo componente sotto carico, ad esempio il Recipe Service durante picchi di richieste all'API OpenAI, senza impattare gli altri servizi.

Isolamento dei guasti

Un malfunzionamento del Recipe Service non pregiudica la disponibilità delle funzionalità di gestione inventario o autenticazione. L'isolamento riduce il rischio di propagazione degli errori all'intero sistema.

Flessibilità tecnologica

Ogni servizio può adottare il database o la libreria più adatta al proprio dominio, come illustrato nella sezione sulla scelta dei database. Nel nostro caso, il Fridge Service utilizza sia PostgreSQL sia MongoDB a seconda della natura dei dati gestiti.

Deploy e aggiornamento indipendenti

La pipeline CI/CD è configurata per rilevare quali servizi sono stati modificati e ricostruire solo i container interessati, riducendo i tempi di deploy e il rischio di regressioni.

Sicurezza per compartimentazione

Il Gateway costituisce l'unico punto di accesso autenticato dall'esterno. I microservizi interni non sono esposti direttamente a Internet e non devono gestire la verifica dei token, riducendo la superficie di attacco.

3.2 Scelta dei Database: SQL e NoSQL

Una delle decisioni architetturali più rilevanti del progetto riguarda l'utilizzo *combinato* di un database relazionale e di uno documentale. Questa scelta non è arbitraria, ma deriva dall'analisi della natura dei dati gestiti dal sistema.

3.2.1 PostgreSQL – Dati Relazionali

PostgreSQL è stato scelto per i dati con struttura fissa e con forti relazioni tra le entità:

- **Utenti (USERS)**: ogni utente ha attributi fissi (ID Firebase, username univoco, email, data di creazione). La struttura non varia tra un utente e l'altro.
- **Frigoriferi (FRIDGES)**: ogni frigorifero ha un proprietario definito (chiave esterna verso **USERS**) e attributi stabili.
- **Membri (FRIDGE_MEMBERS)**: la relazione multi-a-molti tra utenti e frigoriferi richiede integrità referenziale, gestione di ruoli e vincoli di unicità su coppie (`fridge_id`, `user_id`), tutte caratteristiche native dei database relazionali.
- **Inviti (FRIDGE_INVITATIONS)**: i token di invito hanno una scadenza precisa e sono associati a un frigorifero e a un utente creatore; le transazioni ACID garantiscono che un token non possa essere accettato due volte.

La scelta di PostgreSQL per questi dati è motivata dalla necessità di **integrità referenziale**, **transazionalità ACID**, **query complesse con JOIN** e **consistenza forte**, proprietà fondamentali quando si gestiscono permessi e relazioni tra entità.

3.2.2 MongoDB – Dati Eterogenei e Flessibili

MongoDB è stato scelto per la gestione dell’inventario degli alimenti. La motivazione principale è la **variabilità dello schema**: un alimento può avere attributi molto diversi a seconda della sua natura. Ad esempio:

- Un liquido (latte, olio) avrà **unit**: "litri" e una **quantity** decimale.
- Un alimento confezionato può avere **brand**, **notes**, o informazioni nutrizionali.
- I campi `sharing_status.is_common_use` e `sharing_status.is_available_for_loan` formano un oggetto annidato che sarebbe artificialmente normalizzato in un modello relazionale.
- Il modulo di generazione ricette beneficia di un documento ricco e denormalizzato: l’LLM riceve l’intero oggetto alimento come contesto, senza necessità di eseguire JOIN per raccogliere attributi sparsi tra più tabelle.

MongoDB consente inoltre di aggiungere attributi agli alimenti in future iterazioni (allergie, valori calorici, paese di origine) senza richiedere migrazioni dello schema, rendendolo ideale per un sistema in evoluzione agile.

3.3 Toolchain e Motivazioni

Next.js (Frontend)

Framework React di produzione che supporta SSR, SSG e API Routes. La scelta è motivata dalla necessità di avere rendering ottimizzato per SEO, tempi di caricamento ridotti grazie al pre-rendering, e la possibilità di co-locare logica lato server e lato client nello stesso progetto. Il supporto nativo a Tailwind CSS e alle API Routes semplifica il prototipaggio rapido.

Express.js (Backend)

Framework minimalista e maturo per Node.js, scelto per la sua semplicità di configurazione, l’ecosistema di middleware consolidato e la coerenza con il linguaggio JavaScript già utilizzato nel frontend. La versione 5.x adottata introduce la gestione nativa delle promise nei route handler, riducendo il boilerplate per la gestione degli errori asincroni.

Firebase Authentication

Delegare l’autenticazione a Firebase elimina la necessità di implementare e mantenere un sistema di gestione delle credenziali, della rotazione dei token e del

recupero password. Firebase offre provider OAuth integrati (Google, email/password), SDK client per il frontend e SDK Admin per la verifica lato server nel Gateway, con un'affidabilità e una sicurezza superiori a qualsiasi implementazione custom.

Docker e Docker Compose

La containerizzazione garantisce la riproducibilità dell'ambiente tra macchine di sviluppo diverse e semplifica il deploy in produzione. Docker Compose orchestra l'avvio coordinato di tutti i servizi in locale, mentre le immagini multi-stage riducono la dimensione finale dei container escludendo le dipendenze di sviluppo.

GitHub Actions (CI/CD)

La pipeline di integrazione e deploy continuo è implementata tramite GitHub Actions. Il workflow rileva automaticamente quali servizi sono stati modificati (tramite `paths-filter`), esegue lint e test su ciascuno, costruisce le immagini Docker e le pubblica sul Registry remoto. L'automazione riduce il rischio di errori manuali e garantisce che il codice in produzione sia sempre testato.

ESLint (Analisi Statica)

ESLint è integrato in ogni servizio con la configurazione specifica per Node.js/Next.js, garantendo uniformità dello stile del codice tra i membri del team e rilevando errori comuni come variabili non definite, import mancanti o anti-pattern JavaScript.

Jest e Supertest (Analisi Dinamica)

Jest è il framework di testing de facto per l'ecosistema JavaScript/Node.js, con supporto nativo ai mock, alla generazione di report di coverage e all'integrazione con CI. Supertest consente di testare le API Express senza avviare un server reale, simulando richieste HTTP in memoria con pieno controllo sulle asserzioni delle risposte.

OpenAI GPT-4o-mini (Generazione Ricette)

Il modello GPT-4o-mini è stato scelto per il bilanciamento ottimale tra capacità generative, velocità di risposta e costo per token. La generazione di ricette è un task creativo che non richiede la massima potenza computazionale, rendendo un modello di dimensioni ridotte sufficiente e più economico per un progetto accademico.

GitHub (Versionamento)

Il versionamento è gestito tramite GitHub con un flusso strutturato basato su

branch per feature (feature branches), **Pull Request** con revisione obbligatoria prima del merge su **main**, e **Issue** per il tracciamento dei work item di ogni iterazione. I tag di versione identificano i rilasci al termine di ogni sprint. Questo flusso simula le pratiche collaborative dei contesti lavorativi reali.

Postman (Test API Manuali)

Postman è stato utilizzato per il testing esplorativo delle API, in particolare per i componenti che richiedono dipendenze esterne reali (come il Gateway con Firebase) per cui il mock sarebbe eccessivamente complesso. Le collection Postman documentano i casi di test e possono essere eseguite come regression test manuali.

3.4 Versionamento e Gestione dei Microservizi

Ogni microservizio è versionato indipendentemente tramite il campo **version** nel proprio **package.json** e identificato univocamente nel registro container da un tag corrispondente all'hash del commit Git (**:sha**), oltre al tag **:latest**. Questo schema consente il rollback preciso a una versione specifica di un singolo servizio senza dover ricostruire l'intero sistema.

Il repository GitHub è organizzato come **monorepo**: tutti i servizi risiedono nella stessa repository ma in directory separate, semplificando la gestione delle dipendenze condivise, la revisione incrociata del codice e la tracciabilità delle issue. La pipeline CI/CD sfrutta il rilevamento delle modifiche per evitare rebuild non necessari.

4. Metodologia di Sviluppo

4.1 Approccio Agile e Framework Scrum

Per lo sviluppo del progetto è stato adottato un approccio **agile**, nello specifico il framework **Scrum**. Questo approccio si differenzia dai modelli tradizionali come il Waterfall, che prevedono uno sviluppo lineare e rigidamente strutturato, favorendo invece un approccio iterativo e incrementale.

Ogni iterazione rappresenta un ciclo di sviluppo completo e autocontenuto, comprensivo di pianificazione, analisi, progettazione, implementazione, test e documentazione. Al termine di ogni sprint viene rilasciato un incremento funzionale del software, consentendo di valutare i progressi e di adattare le priorità in base ai risultati ottenuti.

I pilastri dello Scrum adottati nel progetto sono:

- **Sprint pianificati:** ogni iterazione ha una durata definita con obiettivi chiari stabiliti all'inizio del ciclo.
- **Backlog prioritizzato:** i requisiti sono catalogati e prioritizzati secondo tre livelli (alta, media, bassa), garantendo che le funzionalità core vengano sviluppate prima delle funzionalità accessorie.
- **Team multifunzionale e auto-organizzato:** ogni membro del team partecipa sia alla definizione dei requisiti sia all'implementazione, eliminando la separazione rigida tra ruoli.
- **Review continua:** al termine di ogni iterazione vengono prodotti diagrammi UML aggiornati, test di copertura e documentazione delle API per validare il lavoro svolto.

4.2 Agile Model Driven Development (AMDD)

Accanto a Scrum è stato adottato l'**AMDD** (*Agile Model Driven Development*), un approccio che integra la modellazione UML nel ciclo agile senza imporre la produzione di documentazione esaustiva prima dell'implementazione.

In conformità con i principi AMDD, i modelli vengono creati *just enough* e *just in time*: si produce solo la documentazione necessaria per comprendere meglio un componente o per facilitare la comunicazione nel team, e lo si fa quando serve, non in anticipo su tutta la progettazione. Nel progetto questo si è tradotto in:

- Diagrammi dei casi d'uso sviluppati nell'Iterazione 0 per definire i requisiti ad alto livello.
- Diagrammi di sequence e component prodotti *durante* o *subito prima* dell'implementazione, come supporto alla progettazione delle API.
- Class diagram dei tipi di dato utilizzati come contratto tra frontend e backend.
- Diagrammi aggiornati ad ogni iterazione per riflettere l'evoluzione del sistema.

4.3 DevOps e Integrazione Continua

Il processo di sviluppo è supportato da una pipeline **DevOps** automatizzata tramite GitHub Actions. Ad ogni push sul branch `main`, il workflow esegue automaticamente le seguenti fasi:

1. **Rilevamento delle modifiche:** identifica quali servizi sono stati toccati dal commit per evitare rebuild inutili.
2. **Analisi della qualità:** per ogni servizio modificato, esegue la verifica delle dipendenze circolari (Madge), l'analisi statica (ESLint) e i test unitari (Jest).
3. **Build e pubblicazione:** costruisce le immagini Docker multi-stage e le pubblica sul Registro remoto con tag `:latest` e `:sha`.

Questa automazione garantisce che ogni modifica sul branch principale sia validata prima di essere distribuita, riducendo il rischio di regressioni e semplificando il processo di deploy.

5. Analisi del Codice

Per garantire la qualità e l'affidabilità del software, è stata effettuata un'attività di analisi del codice suddivisa in due tipologie: **analisi statica** e **analisi dinamica**. L'utilizzo combinato di queste due tecniche consente di individuare errori a diversi livelli: l'analisi statica rileva problemi strutturali senza eseguire il codice, mentre l'analisi dinamica verifica il comportamento effettivo del sistema durante l'esecuzione.

5.1 Analisi Statica

L'analisi statica è effettuata tramite **ESLint**, configurato con le regole specifiche per Node.js e Next.js. Lo strumento analizza automaticamente il codice JavaScript alla ricerca di:

- Errori di sintassi e variabili non definite.
- Violazioni delle convenzioni di stile adottate dal team.
- Pattern potenzialmente problematici (uso di `==` invece di `===`, variabili dichiarate ma non utilizzate, ecc.).
- Dipendenze circolari tra moduli, rilevate tramite la libreria **Madge** integrata nella pipeline CI.

ESLint è integrato sia nell'ambiente di sviluppo locale (con feedback in tempo reale nell'editor) sia nella pipeline CI, dove un'analisi fallita blocca il processo di build.

5.2 Analisi Dinamica

L'analisi dinamica è realizzata tramite **Jest** e **Supertest**. I test sono applicati principalmente agli endpoint delle API, che rappresentano la logica centrale dell'applicazione.

Jest è stato scelto per:

- La facilità di integrazione con Node.js e Next.js.

- Il sistema di mock integrato, che consente di sostituire dipendenze esterne (database, servizi terzi) con oggetti controllati durante i test.
- La generazione automatica di report di coverage (HTML e LCOV), che mostrano la percentuale di codice coperta dai test.
- Il rilevamento automatico dei file di test nella cartella `__tests__`.

Supertest estende Jest consentendo di simulare richieste HTTP verso le applicazioni Express senza avviare un server reale su una porta di rete, rendendo i test più veloci e indipendenti dall'ambiente.

6. Iterazione 0

6.1 Analisi dei Requisiti

L'Iterazione 0 costituisce la fase di avvio del progetto, in cui vengono definiti i requisiti del sistema, analizzati i casi d'uso e progettata l'architettura di alto livello. Non viene prodotto codice funzionante in questa fase; l'obiettivo è stabilire una base solida per le iterazioni successive.

L'analisi dei requisiti ha preso avvio dall'identificazione degli scenari di utilizzo reali del sistema, considerando i diversi profili di utenza (coinquilini, familiari, residenti universitari) e le esigenze specifiche di ciascuno. I requisiti emersi sono stati poi classificati per priorità al fine di orientare la pianificazione degli sprint.

6.1.1 Gestione Account e Accesso

L'accesso all'applicazione è riservato esclusivamente a utenti registrati. Il sistema supporta due modalità di registrazione e autenticazione: tramite form con email e password, oppure tramite provider OAuth esterno (Google). L'autenticazione è gestita da Firebase, che rilascia un token JWT verificato dal Gateway ad ogni richiesta. Ogni utente ottiene uno username univoco che ne consente l'identificazione all'interno dei frigoriferi condivisi.

6.1.2 Gestione Frigorifero

Gli utenti registrati possono creare uno o più frigoriferi virtuali. Il creatore assume automaticamente il ruolo di *Admin del Frigo* (OWNER), con privilegi avanzati quali la modifica delle impostazioni, la gestione dei membri e la cancellazione del frigorifero. Gli altri membri hanno il ruolo di MEMBER.

6.1.3 Gestione Membri e Inviti

L'Admin del Frigo può generare un link di invito temporaneo (valido 24 ore) che consente a qualsiasi utente registrato di unirsi al frigorifero. Il sistema di inviti tramite token garantisce che non siano necessarie autorizzazioni esplicite per ogni singolo invito, semplificando il processo di onboarding.

6.1.4 Gestione Inventario

Tutti i membri possono aggiungere alimenti all'inventario. Il sistema tiene traccia del proprietario di ciascun alimento, della data di scadenza, della categoria, della quantità e di eventuali note. Gli alimenti possono essere marcati come *di uso comune*, rendendoli accessibili a tutti i membri per le ricette condivise.

6.1.5 Supporto alla Creazione di Ricette

Il modulo di generazione ricette integra un LLM (OpenAI GPT-4o-mini) per suggerire ricette basate sugli ingredienti selezionati dall'utente. Le ricette possono essere personali (solo ingredienti dell'utente) o condivise (ingredienti di tutti i membri). Al termine, gli ingredienti utilizzati vengono rimossi dall'inventario. Un algoritmo di equità garantisce una distribuzione bilanciata del contributo degli alimenti tra gli utenti nelle ricette condivise.

6.2 Use Case Diagram



Figura 6.1: Diagramma dei Casi d'Uso – Visione Completa del Sistema

6.3 Analisi delle Specifiche e Prioritizzazione

I requisiti identificati sono stati classificati in tre livelli di priorità, seguendo il principio agile di concentrare gli sforzi iniziali sulle funzionalità essenziali per il funzionamento del sistema (*MVP – Minimum Viable Product*).

6.3.1 Alta Priorità

Le funzionalità di alta priorità costituiscono il nucleo del sistema. Senza di esse l'applicazione non sarebbe utilizzabile e pertanto vengono sviluppate nella prima iterazione.

Funzionalità	Codice UC
Visualizzazione Schermata Iniziale	UC1
Registrazione Utente	UC1.1
Login Utente	UC1.2
Logout Utente	UC1.3
Visualizzazione Dashboard	UC2
Creazione Frigorifero	UC2.1
Visualizzazione Inventario	UC3
Aggiungi Alimento	UC3.1
Rimozione Alimento	UC3.2
Invito al Frigorifero	UC3.3
Cancellazione Frigorifero	UC3.4

6.3.2 Media Priorità

Le funzionalità di media priorità arricchiscono l'esperienza utente e introducono le caratteristiche collaborative avanzate del sistema. Vengono sviluppate nella seconda iterazione.

Funzionalità	Codice UC
Alimenti di Uso Comune	UC5
Tracciamento Proprietario Alimento	UC6
Richiesta Ricetta Personale	UC7
Ricetta Condivisa	UC8
Algoritmo di Sorting	UC9
Algoritmo di Equità	UC10
Gestione Uscita Volontaria da Frigo	UC11
Rimozione Utente da Frigo	UC12

6.3.3 Bassa Priorità

Funzionalità accessorie o migliorative, non sviluppate nelle iterazioni correnti ma incluse nel backlog per sviluppi futuri.

Funzionalità

Prestito Alimenti

Sistema di Notifiche Push

Cronologia Consumi

Preferenze Alimentari e Allergie

Supporto Multilingua

Statistiche Avanzate sui Consumi

6.4 Analisi dell'Architettura – Deployment Diagram

In questa fase viene progettata l'architettura di deployment del sistema, identificando i nodi fisici/logici e le loro interazioni. Il diagramma è volutamente ad alto livello poiché alcune scelte tecnologiche (es. tipo di database) sono ancora da definire.

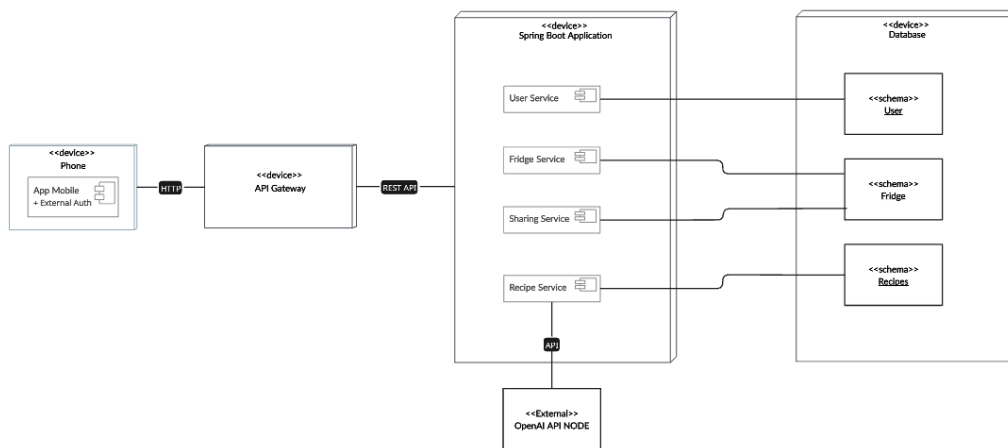


Figura 6.2: Diagramma di Deployment – Iterazione 0

Note: Nel primo nodo (da sinistra) è indicato **App Mobile + External Auth**, poiché la registrazione e l'autenticazione vengono gestite tramite servizio esterno (Firebase). Il **Recipe Service** è collegato a un blocco esterno per la generazione delle ricette tramite LLM. Il database è indicato genericamente come *DataBase*, in attesa della scelta

definitiva sulla tecnologia da adottare (definita come PostgreSQL + MongoDB nell'Iterazione 1).

7. Iterazione 1

7.1 Introduzione

La prima iterazione ha come obiettivo la realizzazione del nucleo funzionale dell'applicazione, ovvero l'insieme minimo di funzionalità senza le quali il sistema non sarebbe utilizzabile. I work item selezionati per questo sprint corrispondono a tutti i casi d'uso di alta priorità identificati nell'Iterazione 0, e seguono una dipendenza logica precisa: prima di poter gestire frigoriferi e alimenti, è necessario disporre di un sistema di autenticazione funzionante; prima di interagire con l'inventario, è necessario che un frigorifero esista e che l'utente ne faccia parte.

Il percorso di sviluppo parte quindi dall'autenticazione (**UC1**), che abilita l'accesso all'applicazione tramite registrazione, login con email/password o provider Google, e logout. Una volta autenticato, l'utente accede alla **Dashboard** (**UC2**), punto centrale di navigazione da cui è possibile creare nuovi frigoriferi (**UC2.1**) e accedere a quelli esistenti. Dall'inventario di un frigorifero (**UC3**) l'utente può aggiungere e rimuovere alimenti (**UC3.1**, **UC3.2**), invitare altri utenti tramite link temporaneo (**UC3.3**) ed eliminare l'intero frigorifero se ne è il proprietario (**UC3.4**).

Caso d'uso	Funzionalità
UC1	Visualizzazione Schermata Iniziale
UC1.1	Registrazione Utente
UC1.2	Login Utente
UC1.3	Logout Utente
UC2	Visualizzazione Dashboard
UC2.1	Creazione Frigorifero
UC3	Visualizzazione Inventario
UC3.1	Aggiunta Alimento
UC3.2	Rimozione Alimento
UC3.3	Invito al Frigorifero
UC3.4	Cancellazione Frigorifero

Parallelamente all'implementazione, vengono prodotti i diagrammi UML di componenti, sequence e class, viene eseguita l'analisi statica tramite ESLint e quella dinamica tramite Jest e Supertest, e le API vengono documentate e verificate tramite Postman.

7.2 Descrizione dei Casi d'Uso

7.2.1 UC1 – Visualizzazione Schermata Iniziale

Descrizione: All'apertura della web app l'utente visualizza la pagina di benvenuto, che presenta le funzionalità principali dell'applicazione e fornisce i link per accedere alla registrazione e al login. Se l'utente ha già effettuato il login, viene reindirizzato direttamente alla Dashboard.

Attori Coinvolti: Tutti gli utenti (autenticati e non)

Precondizione: Nessuna

Postcondizione: Corretta visualizzazione della Schermata Iniziale

Svolgimento standard:

1. L'utente si collega all'URL della web app.
2. Il sistema verifica se è presente un token di sessione valido.
3. Se non autenticato, viene mostrata la landing page con le opzioni di login e registrazione.

7.2.2 UC1.1 – Registrazione Utente

Descrizione: L'utente non registrato compila il form di registrazione inserendo email, password, nome e cognome, oppure si registra tramite il provider OAuth di Google. Il sistema crea il profilo Firebase e, in caso di successo, persiste il profilo applicativo nel database tramite il User Service.

Attori Coinvolti: Utente non registrato

Precondizione: Utente non già registrato con la stessa email

Postcondizione: Profilo utente creato in Firebase e in PostgreSQL; utente reindirizzato alla Dashboard

Trigger: L'utente clicca il link “*Register*” dalla schermata iniziale o dalla pagina di login

Svolgimento standard:

1. L'utente clicca su “*Register*” e viene reindirizzato alla pagina di registrazione.
2. L'utente inserisce i propri dati nel form oppure utilizza il pulsante “*Accedi con Google*”.
3. Il sistema invia la richiesta a Firebase Authentication.
4. Firebase crea il profilo di autenticazione e restituisce un token JWT.
5. Il sistema invoca il User Service per creare il profilo applicativo in PostgreSQL.
6. L'utente viene reindirizzato alla Dashboard.

Svolgimento alternativo:

1. L'utente inserisce un'email già presente nel sistema.
2. Firebase restituisce un errore di email già in uso.
3. Il sistema notifica l'utente e lo invita a effettuare il login o a utilizzare un'email diversa.

7.2.3 UC1.2 – Login Utente

Descrizione: L'utente registrato inserisce le proprie credenziali (email e password) nel form di login oppure utilizza il provider Google. In caso di credenziali corrette, Firebase restituisce un token JWT che viene conservato localmente e allegato a ogni richiesta successiva verso il Gateway.

Attori Coinvolti: Utente registrato non autenticato

Precondizione: Utente non loggato

Postcondizione: Utente loggato; token JWT memorizzato lato client; reindirizzamento alla Dashboard

Trigger: L'utente clicca il link "*Accedi*" dalla schermata iniziale

Svolgimento standard:

1. L'utente clicca su "*Accedi*" e viene reindirizzato alla pagina di login.
2. L'utente inserisce email e password oppure utilizza "*Continua con Google*".
3. Il sistema invia le credenziali a Firebase Authentication.
4. Firebase verifica le credenziali e restituisce un token JWT.
5. Il client memorizza il token e reindirizza l'utente alla Dashboard.
6. In caso di credenziali errate, il sistema mostra un messaggio di errore e invita a riprovare.

7.2.4 UC1.3 – Logout Utente

Descrizione: L'utente può disconnettersi volontariamente o il sistema effettua il logout automatico allo scadere della sessione Firebase.

Attori Coinvolti: Utente autenticato

Precondizione: Utente loggato

Postcondizione: Token invalidato; utente reindirizzato alla schermata iniziale

Trigger: Clic sul pulsante di logout oppure scadenza del token di sessione

Svolgimento standard:

1. L'utente clicca sull'icona del profilo e seleziona *"Logout"*.
2. Il sistema invoca il metodo di sign-out di Firebase, che invalida il token lato client.
3. L'utente viene reindirizzato alla pagina iniziale.

Svolgimento alternativo (logout automatico):

1. Il token di sessione Firebase scade.
2. Il sistema rileva l'assenza di un token valido alla successiva richiesta autenticata.
3. L'utente viene reindirizzato alla pagina di login.

7.2.5 UC2 – Visualizzazione Dashboard

Descrizione: La Dashboard è la pagina principale per gli utenti autenticati. Mostra tutti i frigoriferi a cui l'utente appartiene (come OWNER o MEMBER) e consente di creare nuovi frigoriferi o accedere all'inventario di quelli esistenti.

Attori Coinvolti: Utente autenticato

Precondizione: Utente loggato

Postcondizione: Corretta visualizzazione della Dashboard con l'elenco dei frigoriferi

Trigger: Login avvenuto con successo oppure navigazione diretta alla route `/dashboard`

Svolgimento standard:

1. L'utente accede alla Dashboard (automaticamente dopo il login o navigando a `/dashboard`).
2. Il frontend invia una richiesta GET autenticata al Gateway per ottenere l'elenco dei frigoriferi dell'utente.
3. Il sistema mostra i frigoriferi con nome, ruolo dell'utente e data di creazione.

7.2.6 UC2.1 – Creazione Frigorifero

Descrizione: L'utente autenticato può creare un nuovo frigorifero virtuale assegnandogli un nome. Il sistema crea il record in PostgreSQL, assegna all'utente il ruolo di OWNER e aggiunge automaticamente l'utente come membro.

Attori Coinvolti: Utente autenticato

Precondizione: Utente loggato

Postcondizione: Nuovo frigorifero creato; utente aggiunto come OWNER; frigorifero visibile nella Dashboard

Trigger: Clic sul pulsante “*Nuovo Frigo*”

Svolgimento standard:

1. L'utente clicca su “*Nuovo Frigo*” nella Dashboard.
2. Il sistema mostra un form per l'inserimento del nome del frigorifero.
3. L'utente inserisce un nome e conferma.
4. Il sistema invia una POST al Gateway, che la instrada al Fridge Service.
5. Il Fridge Service crea il record frigorifero in PostgreSQL e aggiunge l'utente come OWNER.
6. Il nuovo frigorifero appare nella Dashboard.

7.2.7 UC3 – Visualizzazione Inventario

Descrizione: Accedendo a un frigorifero, l'utente visualizza l'inventario degli alimenti presenti, l'elenco dei membri, e ha accesso alle azioni disponibili (aggiunta/rimozione alimenti, invito, gestione del frigo).

Attori Coinvolti: Utente membro del frigorifero

Precondizione: Utente loggato e membro del frigorifero

Postcondizione: Visualizzazione completa dell'inventario e dei dati del frigorifero

Trigger: Clic su un frigorifero nella Dashboard

7.2.8 UC3.1 – Aggiungi Alimento

Descrizione: L'utente inserisce un nuovo alimento nell'inventario del frigorifero. I dati dell'alimento (nome, categoria, quantità, unità, scadenza) vengono salvati in MongoDB; l'ID dell'utente viene automaticamente associato all'alimento come proprietario.

Attori Coinvolti: Utente membro del frigorifero

Precondizione: Utente loggato e membro del frigorifero

Postcondizione: Alimento creato in MongoDB e visibile nell'inventario

Trigger: Clic sul pulsante *“Aggiungi Alimento”*

Svolgimento standard:

1. L'utente clicca su *“Aggiungi Alimento”*.
2. Il sistema mostra un form con i campi: nome, categoria, quantità, unità, data di scadenza, note, stato di condivisione.
3. L'utente compila il form e clicca *“Aggiungi”*.
4. Il frontend invia una POST autenticata al Gateway.
5. Il Fridge Service persiste il documento in MongoDB, associando l'ID utente come owner.
6. L'alimento appare nell'inventario.

7.2.9 UC3.2 – Rimozione Alimento

Descrizione: L'utente può rimuovere un alimento dall'inventario. Di norma ogni utente può eliminare solo i propri alimenti; l'Admin del Frigo può rimuovere qualsiasi alimento.

Attori Coinvolti: Utente proprietario dell'alimento o Admin del Frigo

Precondizione: Utente loggato; alimento presente nell'inventario

Postcondizione: Alimento rimosso dal documento MongoDB

Trigger: Clic sul pulsante "*Elimina Alimento*"

Svolgimento standard:

1. L'utente clicca su un alimento nell'inventario.
2. Il sistema mostra i dettagli dell'alimento con il pulsante "*Elimina Alimento*".
3. L'utente clicca su "*Elimina Alimento*" e conferma.
4. Il Fridge Service rimuove il documento da MongoDB.

7.2.10 UC3.3 – Invito al Frigo

Descrizione: L'Admin del Frigo può generare un link di invito con scadenza a 24 ore. Chiunque abbia il link e sia registrato al sistema può unirsi al frigorifero accedendo all'URL di invito.

Attori Coinvolti: Admin del Frigo (generazione); qualsiasi utente registrato (accettazione)

Precondizione: Utente loggato con ruolo OWNER nel frigorifero

Postcondizione: Token di invito generato in PostgreSQL; l'utente che accetta il link viene aggiunto come MEMBER

Trigger: Clic sul pulsante "*Invita*"

Svolgimento standard:

1. L'utente Admin clicca su "*Invita*" nell'inventario del frigorifero.
2. Il sistema genera un token UUID, lo persiste in `FRIDGE_INVITATIONS` con scadenza a 24 ore, e costruisce l'URL di invito.
3. L'utente condivide il link con i potenziali membri.
4. Il destinatario accede al link `/invite/[token]`, effettua il login se necessario, e clicca "*Accetta Invito*".
5. Il Fridge Service verifica la validità del token, aggiunge l'utente come MEMBER e invalida il token.

7.2.11 UC3.4 – Rimozione Frigorifero

Descrizione: L'Admin del Frigo può eliminare definitivamente il frigorifero, rimuovendo tutti i dati associati (membri, inviti, alimenti) sia da PostgreSQL sia da MongoDB.

Attori Coinvolti: Admin del Frigo (ruolo OWNER)

Precondizione: Utente loggato con ruolo OWNER

Postcondizione: Frigorifero e tutti i dati correlati eliminati; frigorifero rimosso dalla Dashboard

Trigger: Clic sul pulsante "*Elimina Frigorifero*"

Svolgimento standard:

1. L'utente Admin accede all'inventario del frigorifero.
2. L'utente clicca su "*Elimina Frigorifero*" e conferma l'operazione.
3. Il Fridge Service rimuove il record da FRIDGES (con cascade su FRIDGE_MEMBERS e FRIDGE_INVITATIONS) e tutti i documenti degli alimenti da MongoDB.
4. Il frigorifero scompare dalla Dashboard di tutti i membri.

7.3 Component Diagram

Il **Component Diagram** UML rappresenta la struttura architettonica del sistema in termini di componenti software e delle loro interdipendenze. Ogni blocco rappresenta un modulo deployabile (frontend, gateway, microservizi, database), mentre le frecce indicano le dipendenze e i canali di comunicazione tra di essi. Nel contesto di questa iterazione, il diagramma riflette l'architettura completa del sistema, incluse le tecnologie definitivamente scelte per i database.

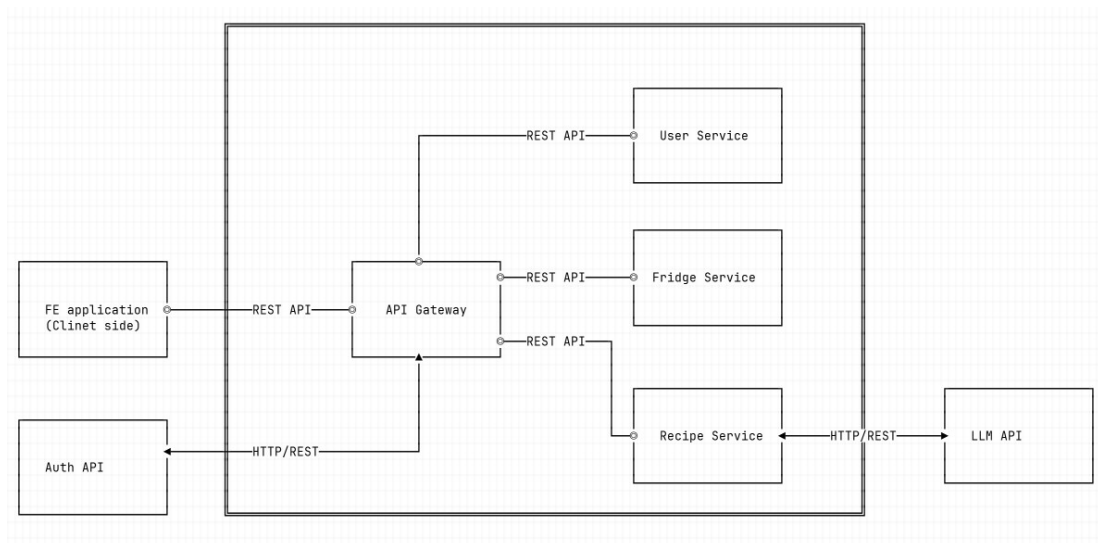


Figura 7.1: Component Diagram – Iterazione 1

7.4 UML Sequence Diagram

I **Sequence Diagram** UML illustrano le interazioni temporali tra i componenti del sistema durante l'esecuzione di una specifica funzionalità. Ogni elemento è rappresentato da una *lifeline* verticale, mentre le frecce orizzontali indicano i messaggi scambiati. Il tempo scorre dall'alto verso il basso. Questi diagrammi sono particolarmente utili per visualizzare il flusso di una richiesta dall'utente fino al database, passando per frontend, gateway e microservizi.

Tool utilizzato per la realizzazione: MonoSketch.

7.4.1 Autenticazione

Il diagramma descrive il flusso di autenticazione dell'utente tramite Firebase. Il client invia le credenziali direttamente all'SDK Firebase, che restituisce un token JWT firmato. Nelle richieste successive, il token viene allegato nell'header **Authorization** e verificato dal Gateway tramite Firebase Admin SDK prima di essere inoltrato ai microservizi. I microservizi non eseguono alcuna verifica autonoma: ricevono le informazioni sull'utente già estratte dal Gateway negli header **X-User-Id** e **X-User-Email**.

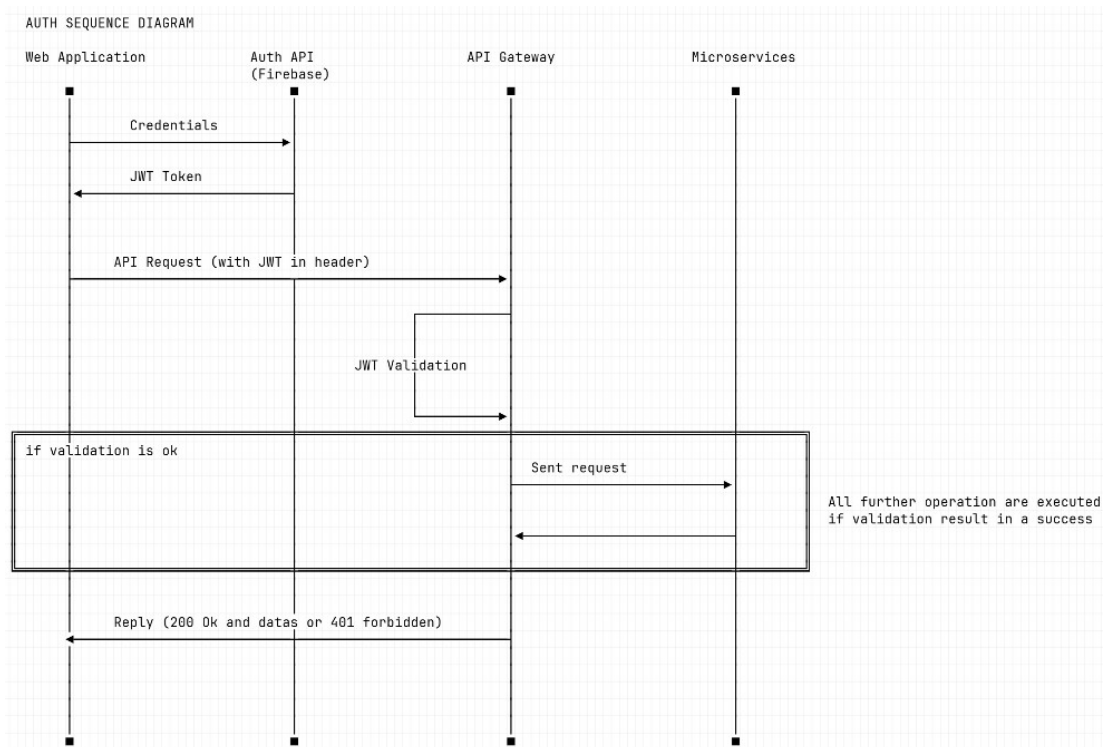


Figura 7.2: Sequence Diagram – Flusso di Autenticazione

7.4.2 Aggiunta di un Alimento

Il diagramma illustra la sequenza di operazioni per l'aggiunta di un nuovo alimento all'inventario. Dopo la verifica del token da parte del Gateway, la richiesta viene instradata al Fridge Service con i dati dell'alimento nel body. Il servizio verifica che l'utente sia effettivamente membro del frigorifero, costruisce il documento MongoDB arricchendolo con il campo `owner_id` (recuperato dall'header `X-User-Id`), e lo persiste nella collection degli alimenti. La risposta include il documento creato con il relativo `_id` generato da MongoDB.

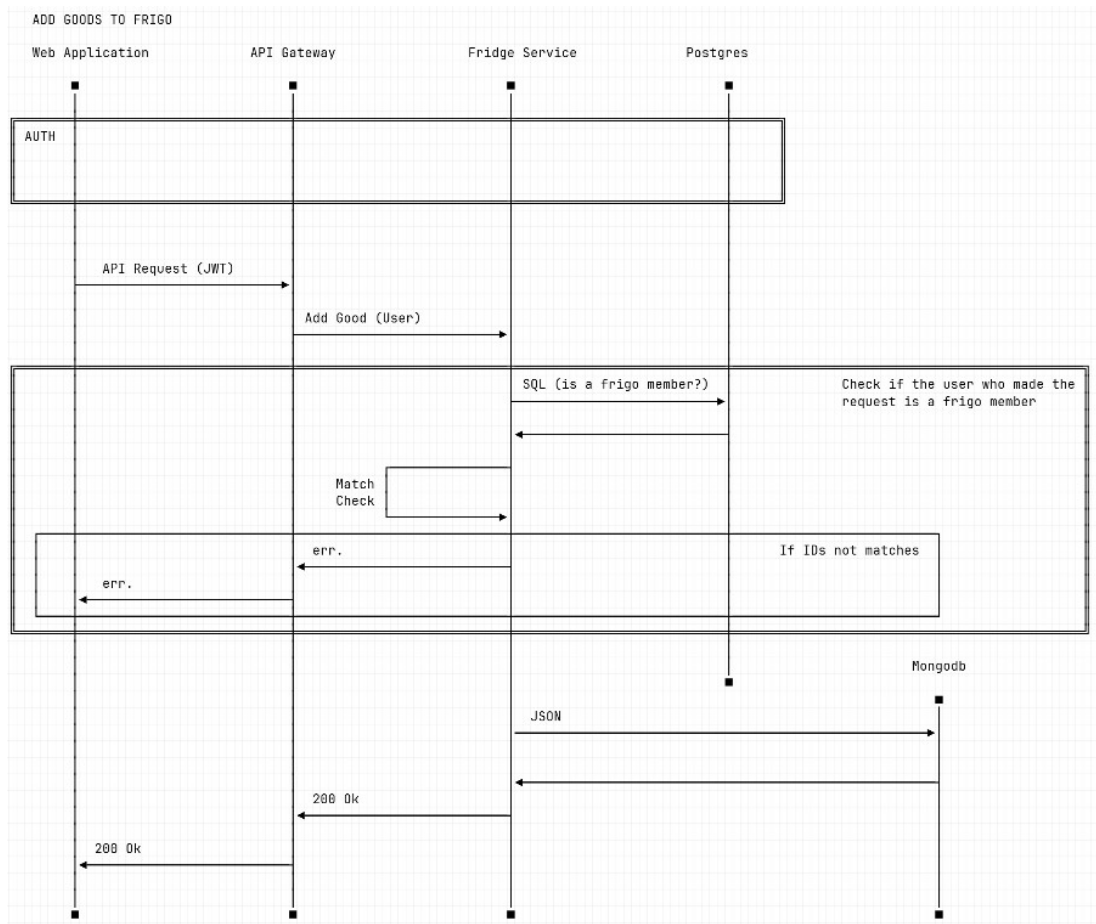


Figura 7.3: Sequence Diagram – Aggiunta Alimento all’Inventario

7.4.3 Creazione Frigorifero

Il diagramma mostra come viene creato un nuovo frigorifero virtuale. Il Fridge Service esegue due operazioni distinte in sequenza: inserisce il record del frigorifero nella tabella **FRIDGES** di PostgreSQL, quindi aggiunge l'utente richiedente nella tabella **FRIDGE_MEMBERS** con il ruolo **OWNER**. L'atomicità delle due operazioni garantisce che non possa esistere un frigorifero senza un proprietario associato. Il nuovo frigorifero è immediatamente visibile nella Dashboard dell'utente.

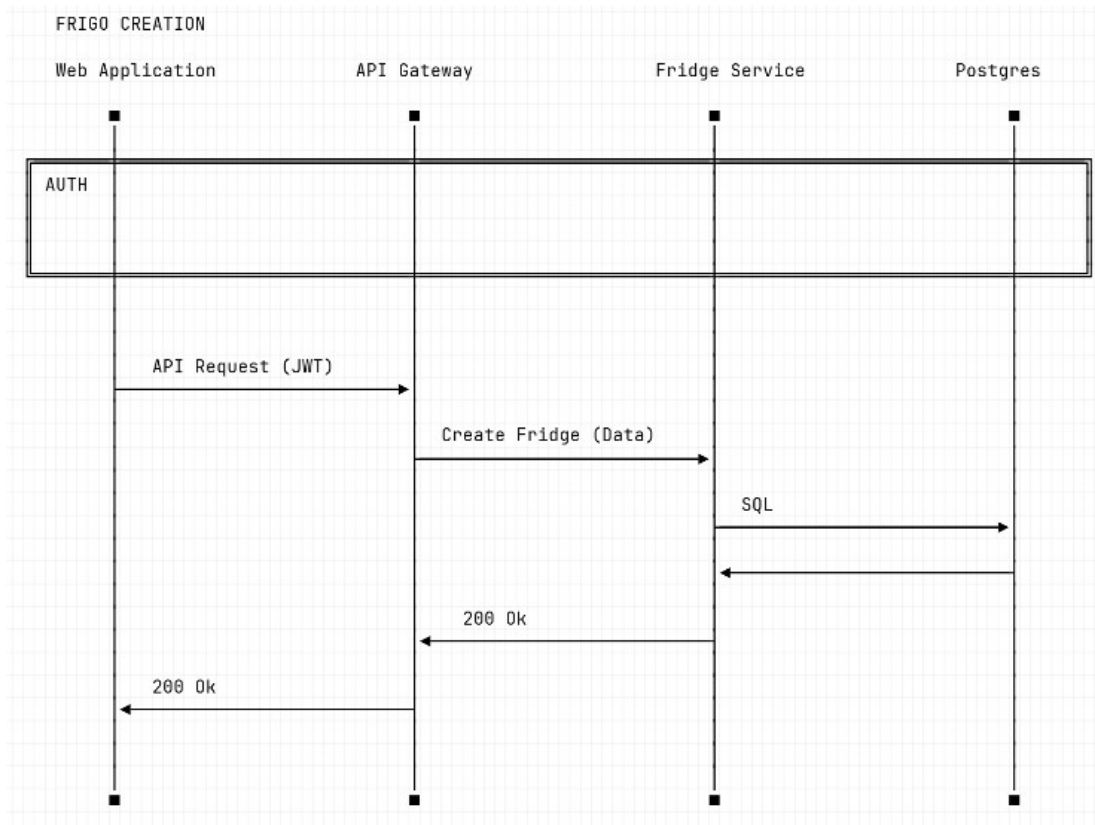


Figura 7.4: Sequence Diagram – Creazione di un Nuovo Frigorifero

7.4.4 Invito Utenti al Frigorifero

Il diagramma descrive il meccanismo di invito basato su token temporanei. L'Admin del Frigo richiede la generazione di un link: il Fridge Service crea un token UUID, lo persiste in `FRIDGE_INVITATIONS` con scadenza a 24 ore, e restituisce l'URL di invito al client. Quando un secondo utente accede al link, il sistema verifica la validità e la non-scadenza del token, aggiunge il nuovo utente come `MEMBER` in `FRIDGE_MEMBERS`, e marca il token come utilizzato per impedirne il riutilizzo.

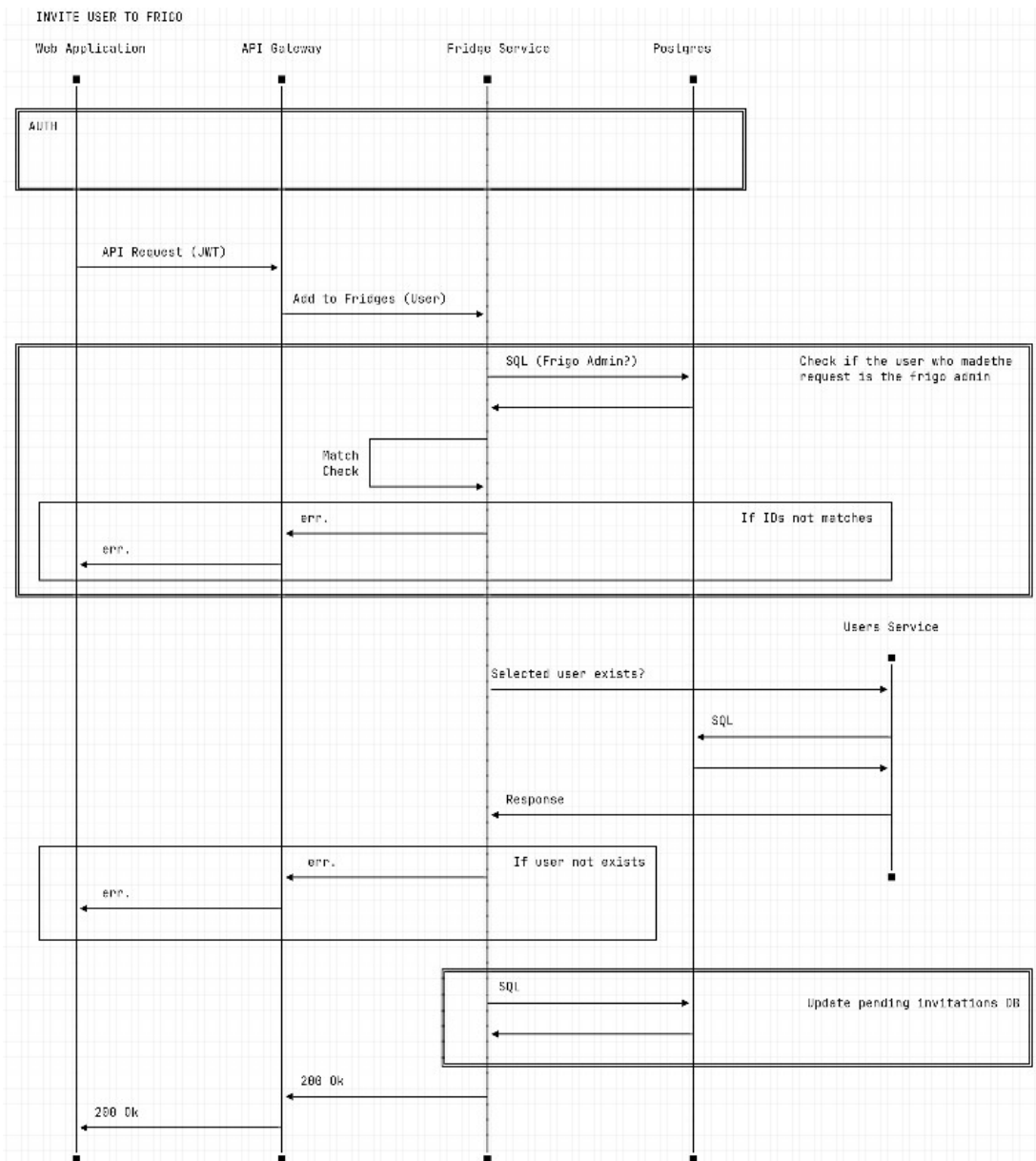


Figura 7.5: Sequence Diagram – Generazione e Accettazione Link di Invito

7.4.5 Gestione Frigorifero (Cancellazione)

Il diagramma illustra le operazioni di cancellazione di un frigorifero, riservata all'utente con ruolo **OWNER**. Prima di eseguire qualsiasi operazione, il Fridge Service verifica il ruolo dell'utente tramite una query su **FRIDGE_MEMBERS**; in caso di ruolo insufficiente risponde con **403 Forbidden**. La cancellazione è un'operazione a cascata: vengono eliminati in sequenza tutti gli alimenti in MongoDB, i record in **FRIDGE_MEMBERS**, i token di invito in **FRIDGE_INVITATIONS** e infine il record del frigorifero in **FRIDGES**.

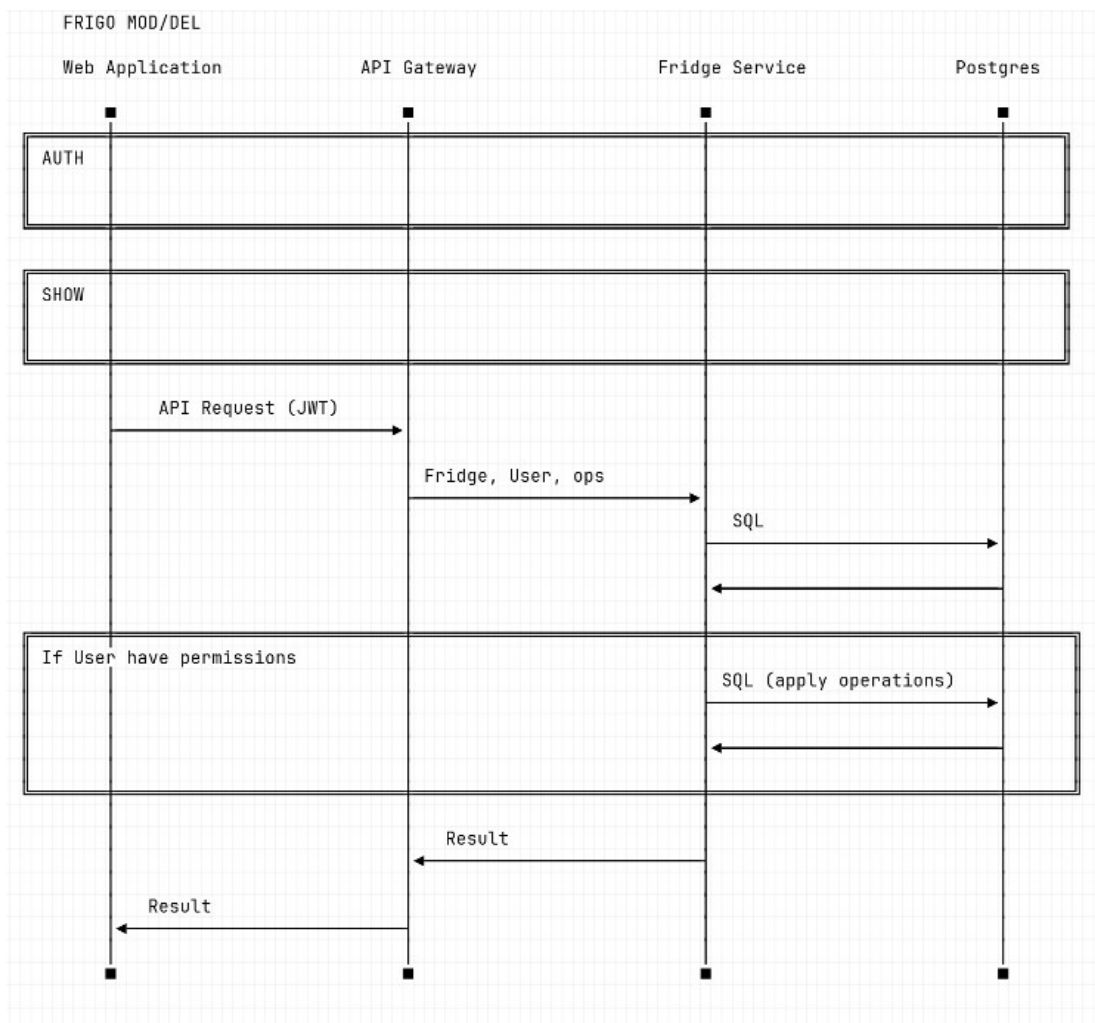


Figura 7.6: Sequence Diagram – Modifica e Cancellazione del Frigorifero

7.4.6 Visualizzazione Alimenti del Frigorifero

Il diagramma descrive il recupero dell'inventario di un frigorifero. Il Fridge Service esegue una query MongoDB filtrando per `fridge_id`, e restituisce la lista dei documenti alimento. Il frontend riceve l'array e popola la vista inventario applicando, lato client, l'ordinamento selezionato dall'utente tramite la funzione `sortIngredients()` (UC9). L'operazione è in sola lettura e non modifica lo stato del sistema.

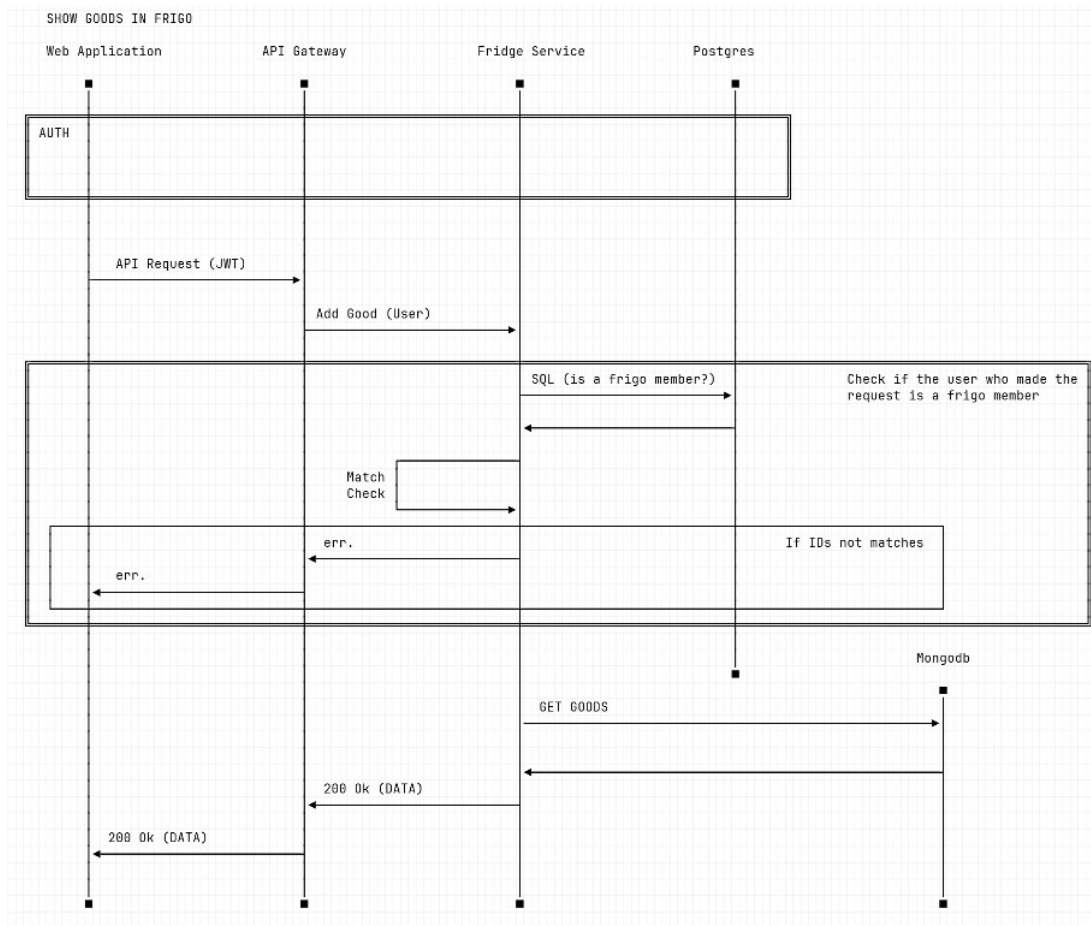


Figura 7.7: Sequence Diagram – Recupero e Visualizzazione degli Alimenti

7.4.7 Visualizzazione Lista Frigoriferi

Il diagramma mostra come la Dashboard recupera i frigoriferi dell'utente autenticato. Il Fridge Service esegue una JOIN tra FRIDGES e FRIDGE_MEMBERS filtrando per `user_id`, e restituisce per ogni frigorifero il nome, la data di creazione e il ruolo dell'utente (OWNER o MEMBER). Questa informazione sul ruolo viene utilizzata dal frontend per mostrare o nascondere le azioni amministrative (modifica, cancellazione, gestione membri) nell'interfaccia utente.

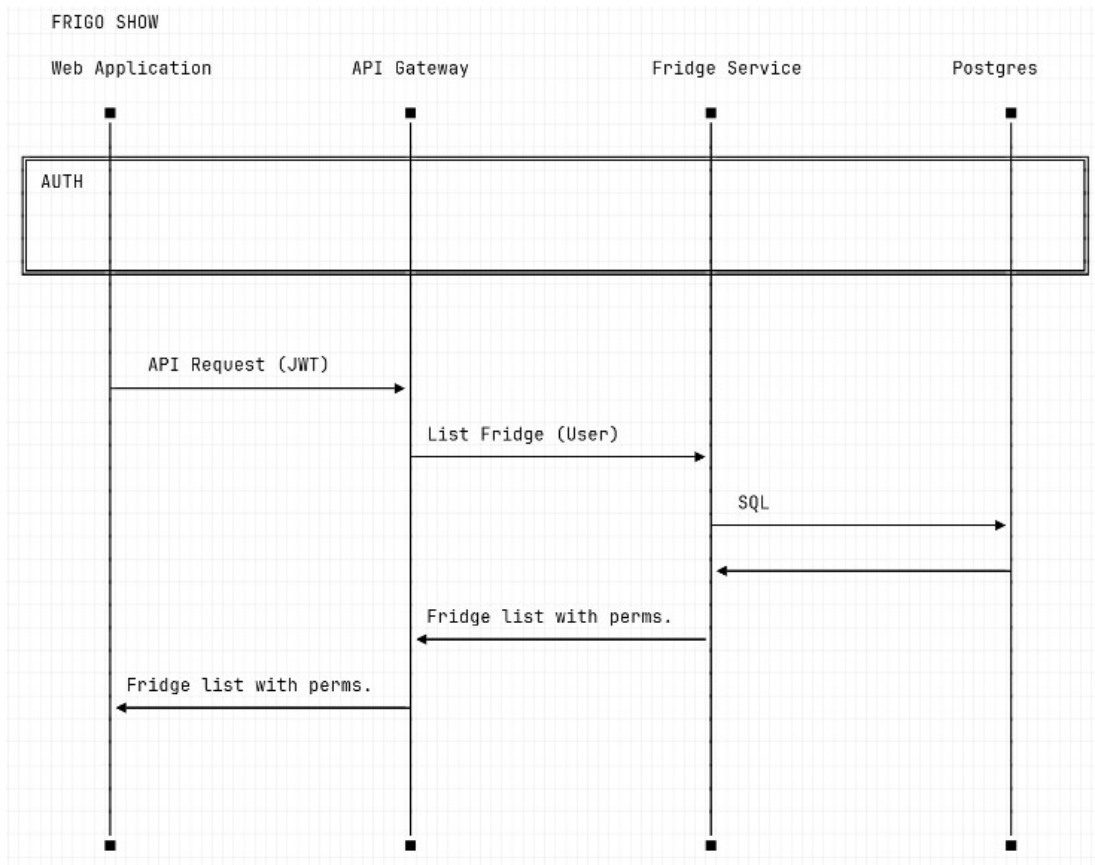


Figura 7.8: Sequence Diagram – Recupero e Visualizzazione dei Frigoriferi nella Dashboard

7.5 UML Class Diagram – Schema dei Dati

Il **Class Diagram** viene qui utilizzato per rappresentare la struttura delle entità dati principali del sistema e le relazioni tra esse. Il modello è suddiviso in due schemi distinti in base alla tecnologia di persistenza:

- **Schema SQL (PostgreSQL)**: per le entità con struttura fissa e relazioni forti, quali `Users`, `Fridges`, `FridgeMembers` e `FridgeInvitations`. Le chiavi esterne e i vincoli di unicità garantiscono l'integrità referenziale.
- **Schema NoSQL (MongoDB)**: per l'entità `Item`, che rappresenta un alimento nell'inventario. La natura eterogenea degli attributi degli alimenti (campo `brand` opzionale, `notes`, oggetto `sharing_status` annidato) rende il documento MongoDB la struttura più adatta. Il modello flessibile facilita inoltre la trasmissione dell'intero documento all'LLM durante la generazione delle ricette, senza necessità di JOIN.

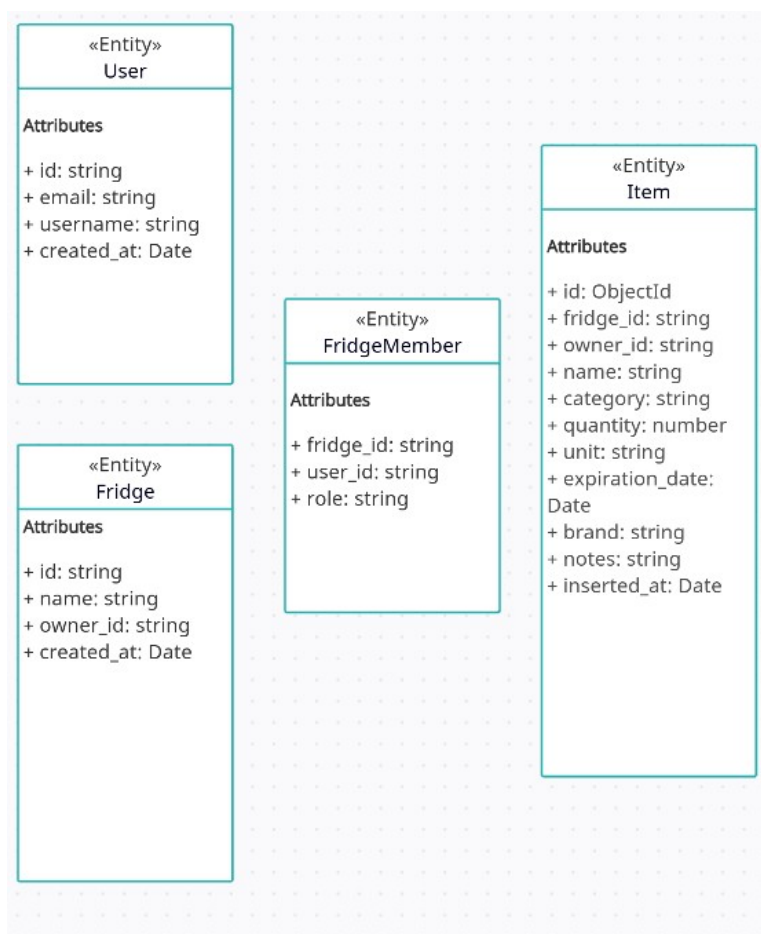


Figura 7.9: Class Diagram – Schema dei Tipi di Dato (SQL e NoSQL)

7.6 Testing

7.6.1 Analisi Statica – ESLint

L'analisi statica è stata condotta tramite ESLint su tutti i servizi del progetto. I report evidenziano la conformità del codice alle regole definite nella configurazione, con indicazione delle eventuali violazioni per file e riga. Durante lo sviluppo dell'Iterazione 1 sono state risolte tutte le segnalazioni di livello *error*, mentre alcune segnalazioni di livello *warning* sono state accettate consapevolmente (es. uso di `console.log` per il debug).

ESLint Report	
0 problems - Generated on Sun Mar 08 2026 17:16:56 GMT+0100 (Ora standard dell'Europa centrale)	
[+] C:\Users\petri\Desktop\Sfrigo\Code\sfrigo\api\fridge-service\index.js	0 problems
[+] C:\Users\petri\Desktop\Sfrigo\Code\sfrigo\api\gateway\authMiddleware.js	0 problems
[+] C:\Users\petri\Desktop\Sfrigo\Code\sfrigo\api\gateway\index.js	0 problems
[+] C:\Users\petri\Desktop\Sfrigo\Code\sfrigo\api\user-service\index.js	0 problems
[+] C:\Users\petri\Desktop\Sfrigo\Code\sfrigo\eslint.config.mjs	0 problems
[+] C:\Users\petri\Desktop\Sfrigo\Code\sfrigo\next.config.mjs	0 problems
[+] C:\Users\petri\Desktop\Sfrigo\Code\sfrigo\postcss.config.mjs	0 problems
[+] C:\Users\petri\Desktop\Sfrigo\Code\sfrigo\src\app\dashboard\fridge\layout.js	0 problems
[+] C:\Users\petri\Desktop\Sfrigo\Code\sfrigo\src\app\dashboard\fridge\page.js	0 problems
[+] C:\Users\petri\Desktop\Sfrigo\Code\sfrigo\src\app\dashboard\layout.js	0 problems
[+] C:\Users\petri\Desktop\Sfrigo\Code\sfrigo\src\app\dashboard\page.js	0 problems
[+] C:\Users\petri\Desktop\Sfrigo\Code\sfrigo\src\app\layout.js	0 problems
[+] C:\Users\petri\Desktop\Sfrigo\Code\sfrigo\src\app\login\layout.js	0 problems
[+] C:\Users\petri\Desktop\Sfrigo\Code\sfrigo\src\app\login\page.js	0 problems
[+] C:\Users\petri\Desktop\Sfrigo\Code\sfrigo\src\app\page.js	0 problems
[+] C:\Users\petri\Desktop\Sfrigo\Code\sfrigo\src\app\register\layout.js	0 problems
[+] C:\Users\petri\Desktop\Sfrigo\Code\sfrigo\src\app\register\page.js	0 problems
[+] C:\Users\petri\Desktop\Sfrigo\Code\sfrigo\src\lib\firebase.js	0 problems
[+] C:\Users\petri\Desktop\Sfrigo\Code\sfrigo\src\middleware.js	0 problems

Figura 7.10: Report di Analisi Statica – ESLint

7.6.2 Analisi Dinamica – Jest e Supertest

Per eseguire l'analisi dinamica sono stati scelti **Jest** e **Supertest**.

Jest è un framework di testing per JavaScript sviluppato da Meta, scelto per la sua facile integrazione con progetti Node.js e Next.js, il sistema di mock integrato che permette di simulare dipendenze esterne (come database e servizi), la generazione automatica dei report di coverage e la configurazione minima, che riconosce automaticamente i file di test nella cartella `__tests__`.

Supertest è una libreria utilizzata per testare applicazioni HTTP Express senza avviare un server reale. Permette di simulare richieste GET, POST e DE-

LETE direttamente in memoria ed è facilmente integrabile con Jest per verificare le risposte delle API in modo semplice e rapido.

Di seguito è riportato un esempio di test relativo al `user-service`:

```
1  const request = require('supertest');
2
3  // Mock del database pg prima di importare il servizio
4  jest.mock('pg', () => {
5    const mPool = {
6      query: jest.fn()
7    };
8    return { Pool: jest.fn(() => mPool) };
9  });
10
11 const { Pool } = require('pg');
12 const pool = new Pool();
13
14 // Importo app
15 const app = require('../api/user-service/index.js');
16
17 describe('User Service - POST /users', () => {
18
19   beforeEach(() => {
20     jest.clearAllMocks(); // resetta i mock prima di ogni test
21   });
22
23   test('TC-US-001 - Ritorna 401 se mancano gli header', async ()
24     => {
25     const res = await request(app)
26       .post('/users')
27       .send({ username: 'testuser' });
28     expect(res.status).toBe(401);
29   });
30
31   test('TC-US-002 - Ritorna 400 se manca username', async () => {
32     const res = await request(app)
33       .post('/users')
34       .set('X-User-Id', 'firebase-uid-123')
35       .set('X-User-Email', 'test@test.com')
36       .send({});
37     expect(res.status).toBe(400);
38   });
39
40   test('TC-US-003 - Ritorna 201 se tutto ok', async () => {
41     pool.query.mockResolvedValueOnce({
```

```

41     rows: [{
42         id: 'firebase-uid-123',
43         username: 'testuser',
44         email: 'test@test.com',
45         created_at: new Date()
46     }]
47 });
48 const res = await request(app)
49     .post('/users')
50     .set('X-User-Id', 'firebase-uid-123')
51     .set('X-User-Email', 'test@test.com')
52     .send({ username: 'testuser' });
53 expect(res.status).toBe(201);
54 expect(res.body.username).toBe('testuser');
55 });
56
57 });

```

Listing 7.1: Test UC – user-service

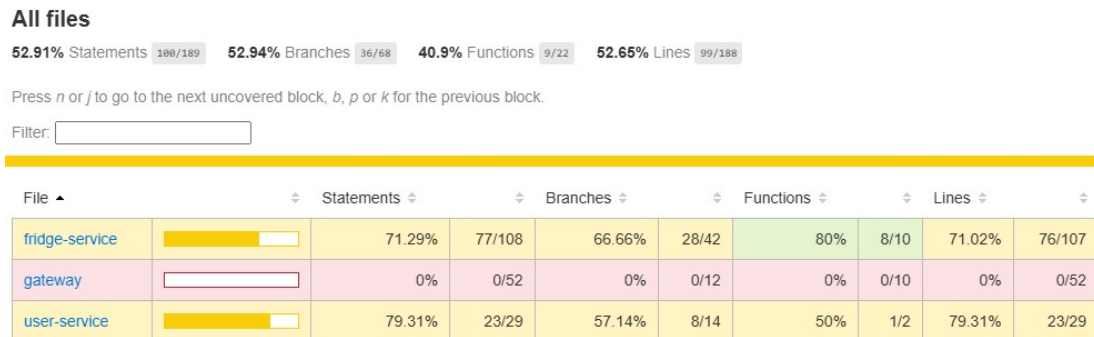


Figura 7.11: Report di Coverage – Jest (Iterazione 1)

Disclaimer: Il Gateway utilizza Firebase Admin SDK per la validazione dei token JWT. Poiché Firebase Authentication non è eseguibile in locale come un normale database, il mock dell'SDK richiederebbe la simulazione dell'intera infrastruttura di autenticazione Google. Per questo motivo il testing del Gateway è stato effettuato manualmente tramite **Postman**, inviando token Firebase reali ottenuti dopo un'autenticazione effettiva. Questo approccio garantisce la verifica del comportamento reale del servizio, a differenza di un mock che potrebbe mascherare problemi di integrazione.

7.7 Documentazione API

Le API sviluppate nell'Iterazione 1 sono state verificate manualmente tramite Postman. Per ogni endpoint vengono riportati il metodo HTTP, il percorso, i parametri principali e la risposta restituita dal sistema.

7.7.1 Add Item e Delete Item

POST `/api/v1/fridges/:fridgeId/items` aggiunge un nuovo alimento all'inventario del frigorifero specificato. Il body JSON deve includere almeno **name**, **category**, **quantity** e **unit**; il campo **owner_id** viene iniettato automaticamente dal Fridge Service tramite l'header **X-User-Id**. In caso di successo restituisce 201 Created con il documento MongoDB creato.

DELETE `/api/v1/fridges/:fridgeId/items/:itemId` rimuove l'alimento identificato da **itemId** dall'inventario. Il servizio verifica che l'utente richiedente sia il proprietario dell'alimento o abbia ruolo OWNER nel frigorifero prima di procedere con la cancellazione; in caso contrario risponde con 403 Forbidden.

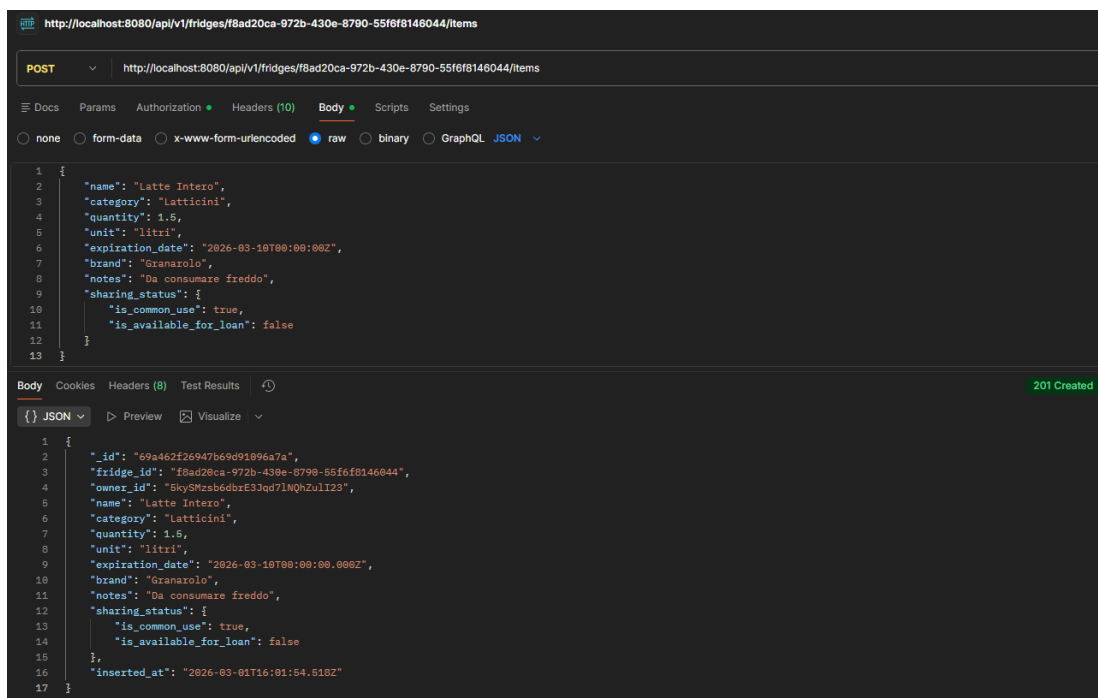


Figura 7.12: POST `/api/v1/fridges/:id/items`

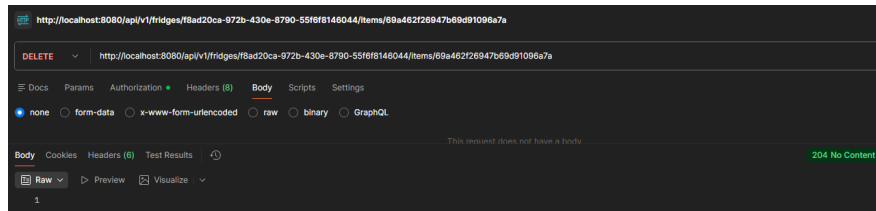


Figura 7.13: DELETE `/api/v1/fridges/:id/items/:itemId`

7.7.2 Get Frigos e Get Items

GET `/api/v1/fridges` restituisce la lista dei frigoriferi a cui l'utente autenticato appartiene, con indicazione del ruolo (OWNER o MEMBER) e della data di creazione. Il Fridge Service esegue una JOIN tra le tabelle FRIDGES e FRIDGE_MEMBERS filtrando per `user_id`.

GET `/api/v1/fridges/:fridgeId/items` restituisce l'inventario completo del frigorifero. Prima di eseguire la query MongoDB, il servizio verifica che l'utente sia membro del frigorifero tramite la tabella FRIDGE_MEMBERS; in caso contrario risponde 403 Forbidden. La risposta è un array di documenti alimento ordinati per data di inserimento.

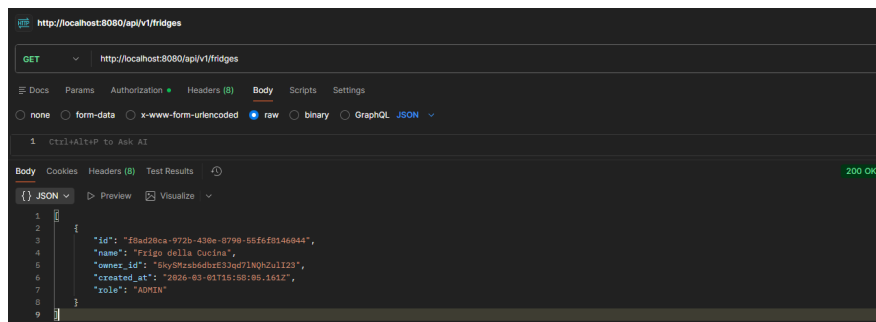


Figura 7.14: GET `/api/v1/fridges`

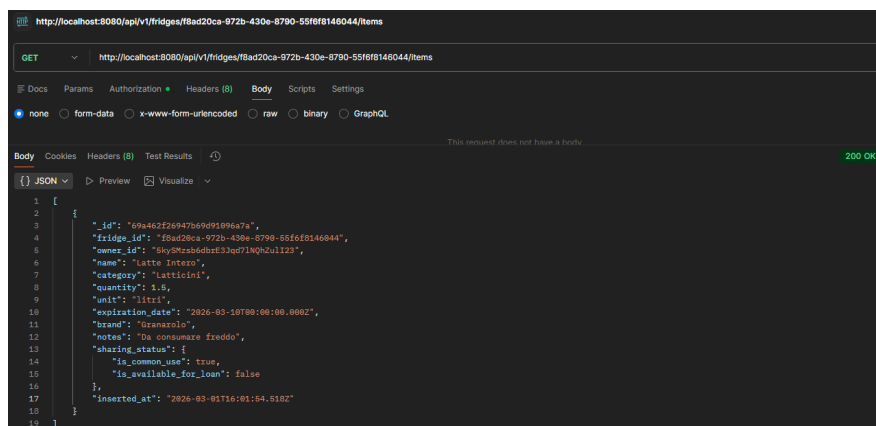


Figura 7.15: GET `/api/v1/fridges/:id/items`

Il login non è gestito da un endpoint REST applicativo, ma direttamente dall'SDK Firebase lato client. Dopo l'autenticazione con email/password o provider Google, Firebase rilascia un token JWT che il frontend allega in ogni richiesta nell'header **Authorization: Bearer <token>**. Il Gateway lo verifica tramite Firebase Admin SDK e, se valido, estrae **uid** e **email** dell'utente per propagarli ai microservizi. Lo screenshot mostra il token JWT ottenuto e la risposta del Gateway a una richiesta autenticata, confermando che la catena di verifica funziona correttamente end-to-end.



8. Iterazione 2

8.1 Introduzione

Con il nucleo funzionale completato nell'Iterazione 1, la seconda iterazione si concentra sulle funzionalità collaborative avanzate che costituiscono il valore differenziale di Sfrigo rispetto a una semplice lista della spesa condivisa. I work item selezionati per questo sprint sono tutti i casi d'uso di media priorità, raggruppabili in tre aree tematiche strettamente interconnesse.

La prima area riguarda la **gestione della condivisione**: UC5 introduce il concetto di alimento di uso comune, mentre UC6 garantisce che ogni alimento sia sempre tracciato al proprio proprietario. Questi due work item sono propedeutici agli algoritmi dei punti successivi, poiché senza la distinzione proprietario/comune non sarebbe possibile calcolare contributi equi né separare ricette personali da condivise.

La seconda area riguarda la **generazione di ricette tramite AI**: UC7 e UC8 implementano rispettivamente la ricetta personale (solo ingredienti dell'utente) e quella condivisa (ingredienti comuni del gruppo). Entrambi si appoggiano al Recipe Service introdotto nell'architettura ma non ancora attivato nell'Iterazione 1. UC9 completa questa area con l'algoritmo di sorting dell'inventario, che permette di ordinare gli alimenti per urgenza di consumo e quindi di orientare la scelta degli ingredienti da utilizzare per primi.

La terza area riguarda la **gestione avanzata dei membri**: UC10 introduce l'algoritmo di equità che calcola il contributo di ciascun membro al frigorifero condiviso, UC11 consente a un utente di abbandonare volontariamente un frigorifero, e UC12 permette all'Admin di rimuovere un membro. In questa iterazione viene inoltre integrata la pipeline **CI/CD** tramite GitHub Actions, automatizzando il ciclo di test, build e pubblicazione delle immagini Docker ad ogni push sul branch `main`.

Caso d'uso	Funzionalità	Area
UC5	Alimento di Uso Comune	Condivisione
UC6	Tracciamento Proprietario Alimento	Condivisione
UC7	Richiesta Ricetta Personale	Ricette AI
UC8	Richiesta Ricetta Condivisa	Ricette AI
UC9	Algoritmo di Sorting	Ricette AI
UC10	Algoritmo di Equità	Gestione membri
UC11	Uscita Volontaria dal Frigorifero	Gestione membri
UC12	Rimozione Utente dal Frigorifero	Gestione membri

8.2 Descrizione dei Casi d'Uso

8.2.1 UC5 – Alimento di Uso Comune

Descrizione: Durante l'inserimento di un alimento, l'utente può marcarlo come *di uso comune*, rendendolo disponibile a tutti i membri del frigorifero per la generazione di ricette condivise. Lo stato di condivisione è memorizzato nel campo `sharing_status.is_common_use` del documento MongoDB.

Attori Coinvolti: Utente membro del frigorifero

Precondizione: Utente loggato; form di aggiunta alimento aperto

Postcondizione: Alimento salvato con flag `is_common_use: true`

Trigger: L'utente spunta la casella "*Alimento Condiviso*" durante l'aggiunta

Svolgimento standard:

1. L'utente segue il flusso di UC3.1 – Aggiungi Alimento.
2. L'utente spunta la casella "*Alimento Condiviso*" nel form.
3. Il sistema salva l'alimento con `is_common_use: true` in MongoDB.
4. L'alimento è ora selezionabile da tutti i membri nelle ricette condivise.

8.2.2 UC6 – Tracciamento Proprietario Alimento

Descrizione: Ad ogni alimento aggiunto viene automaticamente associato il campo `owner_id` con l'identificativo dell'utente che lo ha inserito. Questo avviene in modo trasparente senza richiedere interazioni aggiuntive. Il tracciamento del proprietario è fondamentale per l'algoritmo di equità e per il controllo degli accessi in fase di eliminazione.

Attori Coinvolti: Utente membro del frigorifero (implicitamente)

Precondizione: Utente loggato

Postcondizione: Il documento MongoDB dell'alimento contiene `owner_id` con l'UID dell'utente corrente

Trigger: L'utente esegue UC3.1 – Aggiungi Alimento

8.2.3 UC7 – Richiesta Ricetta Personale

Descrizione: L'utente loggato accede all'inventario di un frigorifero e richiede una ricetta personale tramite il servizio AI. Il sistema genera ricette basandosi sugli ingredienti selezionati dall'utente utilizzando l'API OpenAI. La ricetta viene preparata considerando solo gli ingredienti scelti dall'utente, che al termine vengono rimossi dall'inventario.

Attori Coinvolti: Utente membro del frigorifero

Precondizione: Utente loggato; almeno 2 alimenti personali nell'inventario

Postcondizione: Ricette generate dall'LLM mostrate all'utente; ingredienti selezionati rimossi al momento della scelta

Trigger: Clic sul pulsante "*Ricetta AI*" → selezione "*Ricetta Personale*"

Svolgimento standard:

1. L'utente loggato accede all'inventario del frigo.
2. L'utente clicca il bottone "*Ricetta AI*".
3. Il sistema mostra le opzioni disponibili: "*Ricetta Personale*" e "*Ricetta Condivisa*".
4. L'utente clicca su "*Ricetta Personale*".
5. Il sistema mostra gli ingredienti selezionabili dall'inventario.
6. L'utente seleziona almeno 2 ingredienti.
7. L'utente clicca su "*Cerca Ricette*".
8. Il sistema invia la richiesta all'API OpenAI e genera una lista di ricette suggerite.
9. L'utente visualizza le ricette proposte e sceglie quella desiderata.
10. L'utente clicca su "*Usa questa ricetta*".
11. Il sistema rimuove dall'inventario gli ingredienti selezionati.

8.2.4 UC8 – Richiesta Ricetta Condivisa

Descrizione: L'utente loggato accede all'inventario di un frigorifero condiviso e richiede una ricetta condivisa tramite il servizio AI. A differenza della ricetta personale, la ricetta condivisa considera l'intero inventario del frigo, accessibile da tutti i membri. Gli ingredienti selezionati vengono rimossi dall'inventario al termine.

Attori Coinvolti: Utente membro del frigorifero

Precondizione: Utente loggato; almeno 2 alimenti comuni presenti nell'inventario

Postcondizione: Ricette generate basate sugli alimenti condivisi; ingredienti rimossi con aggiornamento del bilancio di equità

Trigger: Clic sul pulsante "*Ricetta AI*" → selezione "*Ricetta Condivisa*"

Svolgimento standard:

1. L'utente loggato accede all'inventario del frigo.
2. L'utente clicca il bottone "*Ricetta AI*".
3. Il sistema mostra le opzioni disponibili: "*Ricetta Personale*" e "*Ricetta Condivisa*".
4. L'utente clicca su "*Ricetta Condivisa*".
5. Il sistema mostra gli ingredienti selezionabili dall'inventario condiviso.
6. L'utente seleziona almeno 2 ingredienti.
7. L'utente clicca su "*Cerca Ricette*".
8. Il sistema invia la richiesta all'API OpenAI e genera una lista di ricette suggerite.
9. L'utente visualizza le ricette proposte e sceglie quella desiderata.
10. L'utente clicca su "*Usa questa ricetta*".
11. Il sistema rimuove dall'inventario condiviso gli ingredienti selezionati.

8.2.5 UC9 – Algoritmo di Sorting

Descrizione: Il sistema ordina e prioritizza gli alimenti nell'inventario tramite una funzione di sorting che supporta più modalità. La modalità più sofisticata è *urgency*, che implementa un algoritmo di ordinamento basato su uno score composito.

Attori Coinvolti: Utente membro del frigorifero

Precondizione: Utente loggato; inventario non vuoto

Postcondizione: Alimenti visualizzati ordinati secondo il criterio selezionato

Trigger: Selezione di una modalità di ordinamento nel menu dell'inventario

Modalità di ordinamento disponibili:

- **Nome** ($A \rightarrow Z, Z \rightarrow A$): ordinamento alfabetico.
- **Scadenza**: dall'alimento con scadenza più imminente al più lontano.
- **Categoria**: raggruppamento per categoria alimentare.
- **Proprietario**: raggruppamento per utente proprietario.
- **Urgenza**: score composito basato su scadenza, categoria e quantità residua.

Pseudocodice – MergeSort con compareFn:

```
1 sort(compareFn)
2   MergeSort(list, left, right)
3     if left >= right -> return
4     middle = (left + right) / 2
5     MergeSort(list, left, middle)
6     MergeSort(list, middle + 1, right)
7     Merge(list, left, middle, right)
8
9 Merge(list, left, middle, right)
10  // Confronto tramite compareFn(a, b)
11  while i < n1 && j < n2
12    if compareFn(leftArray[i], rightArray[j]) <= 0
13      list[k] = leftArray[i]; i++
14    else
15      list[k] = rightArray[j]; j++
```

Listing 8.1: Algoritmo di Sorting – MergeSort

Complessità:

- Ricorsione e divisione: $O(\log_2 n)$
- Merge: $O(n)$
- **Totale:** $O(n \log n)$

Per la modalità *urgency* è stato adottato uno **score composito**: il 70% del peso è dato da uno score di scadenza e il 30% dalla categoria dell'alimento. La funzione `clamp` mantiene i valori tra 0 e 30, normalizzandoli successivamente:

$$\text{score} = (\text{expiryScore} \times 0.7) + ((1 - \text{categoryWeight}) \times 0.3) \quad (8.1)$$

$$\text{expiryScore} = \frac{\text{clamp}(\text{daysUntilExpiry}, 0, 30)}{30} \quad (8.2)$$

8.2.6 UC10 – Algoritmo di Equità

Descrizione: Il sistema calcola e visualizza la distribuzione equa del contributo alimentare tra i membri del frigorifero condiviso. L'algoritmo considera il valore (stimato o effettivo) degli alimenti inseriti da ciascun utente e calcola quante risorse ciascun membro ha messo a disposizione della collettività, esprimendo il risultato in termini monetari o percentuali.

Attori Coinvolti: Utente membro del frigorifero

Precondizione: Frigorifero con almeno 2 membri; alimenti presenti nell'inventario condiviso

Postcondizione: Visualizzazione del riepilogo contributivo per ogni membro

Trigger: Richiesta ricetta condivisa o accesso alla sezione “Equità”

Pseudocodice – Calcolo dell'Equità:

```
1 GET /fridges/:fridgeId/equity
2
3 // Verifica accesso
4 membership = checkFridgeMembership(fridgeId, userId)
5 if !membership -> return 403
6
7 // Recupera dati
8 members = query PostgreSQL -> fridge_members JOIN users WHERE
   fridge_id
9 recipes = query MongoDB -> recipes WHERE fridge_id
10
11 // Inizializza struttura stats
12 for each member m in members:
13   stats[m.user_id] = {
14     recipes_participated: 0, // R_u
15     ingredients_provided: 0, // C_u
16     total_ingredients: 0, // T_u
17     members_per_recipe: []
18   }
19
20 // Calcola statistiche per ogni ricetta
21 for each recipe in recipes: // O(r)
22   snapshot = recipe.members_snapshot
23   ingredients = recipe.ingredients
```

```

24     memberCount = len(snapshot)
25     totalIngr   = len(ingredients)
26
27     for each member in snapshot:                // 0(m)
28         if member not in stats -> skip
29         stats[member].recipes_participated += 1
30         stats[member].total_ingredients    += totalIngr
31         stats[member].members_per_recipe.push(memberCount)
32
33         provided = count ingredients where owner_id == member // 0
34         (i)
35         stats[member].ingredients_provided += provided
36
37     // Calcola equity index per ogni membro
38     for each user u in stats:                    // 0(u)
39         R = recipes_participated
40         C = ingredients_provided
41         T = total_ingredients
42
43         if R == 0:
44             label = "Nessuna partecipazione"; confidence = 0
45             continue
46
47         contribution_rate = C / T
48         expected_rate     = sum(1/n for n in members_per_recipe) / R
49         equity_index      = contribution_rate / expected_rate
50         confidence        = 1 - e^(-R/3)
51         display_score      = equity_index * confidence
52
53         // Assegna etichetta
54         if display_score >= 1.1 -> "Contribuisce troppo" (lightblue)
55         if display_score >= 0.9 -> "In equilibrio" (green)
56         if display_score >= 0.6 -> "Contribuisce poco" (yellow)
57         if display_score >= 0.3 -> "Contribuisce raramente" (orange)
58         else -> "Non contribuisce" (red)
59
60     // Ordina per display_score decrescente
61     result.sort((a, b) -> b.display_score - a.display_score) // 0(
62     u log u)
63
64     return result

```

Listing 8.2: Algoritmo di Equità – GET /fridges/:fridgeId/equity

L'endpoint `GET /api/v1/fridges/:fridgeId/equity` restituisce il calcolo aggiornato in tempo reale, permettendo ai membri di visualizzare lo stato dei contributi e di bilanciare le future aggiunte di alimenti.

Complessità: il termine dominante è $O(r \cdot m \cdot i)$, corrispondente al loop più annidato. Considerando che un frigorifero difficilmente avrà un numero elevato di membri e ingredienti (es. $m = 10$, $i = 20$), si ottiene:

$$O(r \cdot m \cdot i) \approx O(r \times 10 \times 20) \approx O(r)$$

La **complessità totale** dell'algoritmo è quindi $O(r)$.

8.2.7 UC11 – Uscita Volontaria dal Frigorifero

Descrizione: Un utente membro (non OWNER) può scegliere di abbandonare volontariamente un frigorifero. Al momento dell'uscita, il sistema può richiedere come gestire gli alimenti di proprietà dell'utente (rimuoverli o trasferirli all'OWNER).

Attori Coinvolti: Utente membro (ruolo MEMBER)

Precondizione: Utente loggato; membro del frigorifero con ruolo MEMBER

Postcondizione: Utente rimosso da FRIDGE_MEMBERS; alimenti dell'utente gestiti secondo la scelta

Trigger: Clic sul pulsante “*Lascia Frigorifero*” nelle impostazioni del frigo

8.2.8 UC12 – Rimozione Utente dal Frigorifero

Descrizione: L'Admin del Frigo (OWNER) può rimuovere un membro dal frigorifero. L'operazione è irreversibile e rimuove il record corrispondente dalla tabella FRIDGE_MEMBERS.

Attori Coinvolti: Admin del Frigo (ruolo OWNER)

Precondizione: Utente loggato con ruolo OWNER; membro target presente nel frigorifero

Postcondizione: Membro rimosso da FRIDGE_MEMBERS; il membro espulso non ha più accesso al frigorifero

Trigger: Clic sul pulsante “*Rimuovi Membro*” nella gestione dei membri del frigo

8.3 Sequence Diagram – Iterazione 2

I tre sequence diagram di questa iterazione coprono le funzionalità introdotte nei work item di media priorità. A differenza dei diagrammi dell'Iterazione 1, dove i flussi erano prevalentemente CRUD su entità singole, qui le interazioni coinvolgono più sorgenti dati (PostgreSQL e MongoDB in lettura combinata) e servizi esterni (OpenAI API), rendendo i flussi strutturalmente più complessi.

8.3.1 Generazione Ricetta

Il diagramma illustra il flusso completo della generazione ricetta, comune sia alla modalità personale (UC7) sia a quella condivisa (UC8). L'utente seleziona gli ingredienti desiderati nell'inventario e avvia la richiesta. Il Fridge Service recupera i documenti MongoDB degli ingredienti selezionati e li trasmette al Recipe Service. Quest'ultimo costruisce un prompt strutturato contenente i dati degli ingredienti (nome, categoria, quantità, unità) e lo invia all'API OpenAI GPT-4o-mini tramite una richiesta HTTP POST. L'LLM restituisce un array di ricette in formato JSON; il Recipe Service deserializza la risposta e la restituisce al frontend, che le mostra all'utente. Alla selezione della ricetta da parte dell'utente, il Fridge Service rimuove da MongoDB gli ingredienti utilizzati.

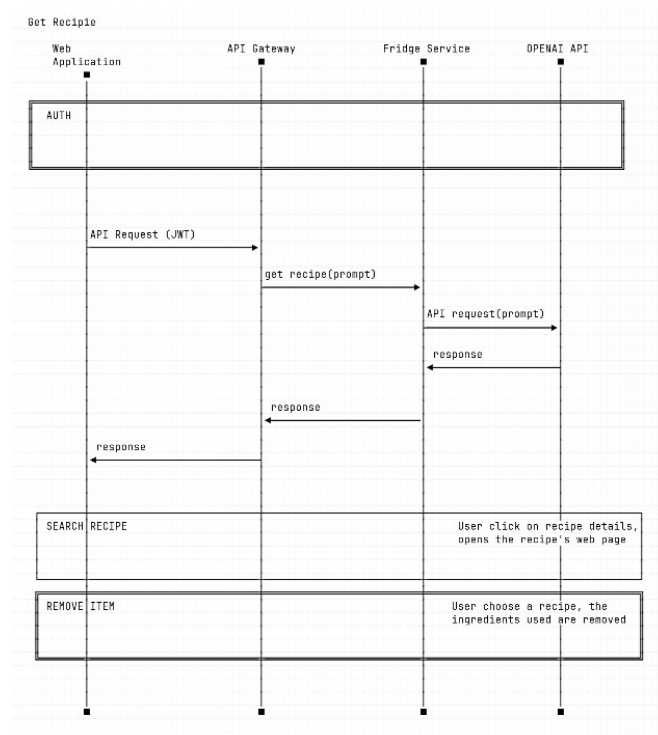


Figura 8.1: Sequence Diagram – Generazione Ricetta tramite LLM (UC7, UC8)

8.3.2 Calcolo Equità

Il diagramma descrive il flusso del calcolo dell'equità (UC10). Alla richiesta `GET /api/v1/fridges/:fridgeId/equity`, il Fridge Service esegue in parallelo due query: una su PostgreSQL per ottenere la lista dei membri del frigorifero con i rispettivi identificativi, e una su MongoDB per recuperare tutti gli alimenti comuni (`is_common_use: true`) con il loro valore stimato. Con questi dati, il servizio aggrega il contributo totale per `owner_id`, calcola la quota ideale per membro (valore totale / numero di membri) e restituisce per ciascun utente il delta rispetto alla quota: un valore positivo indica che ha contribuito più della quota, un valore negativo che è in debito verso il gruppo. Il risultato viene formattato e restituito al frontend per la visualizzazione nella schermata dedicata all'equità.

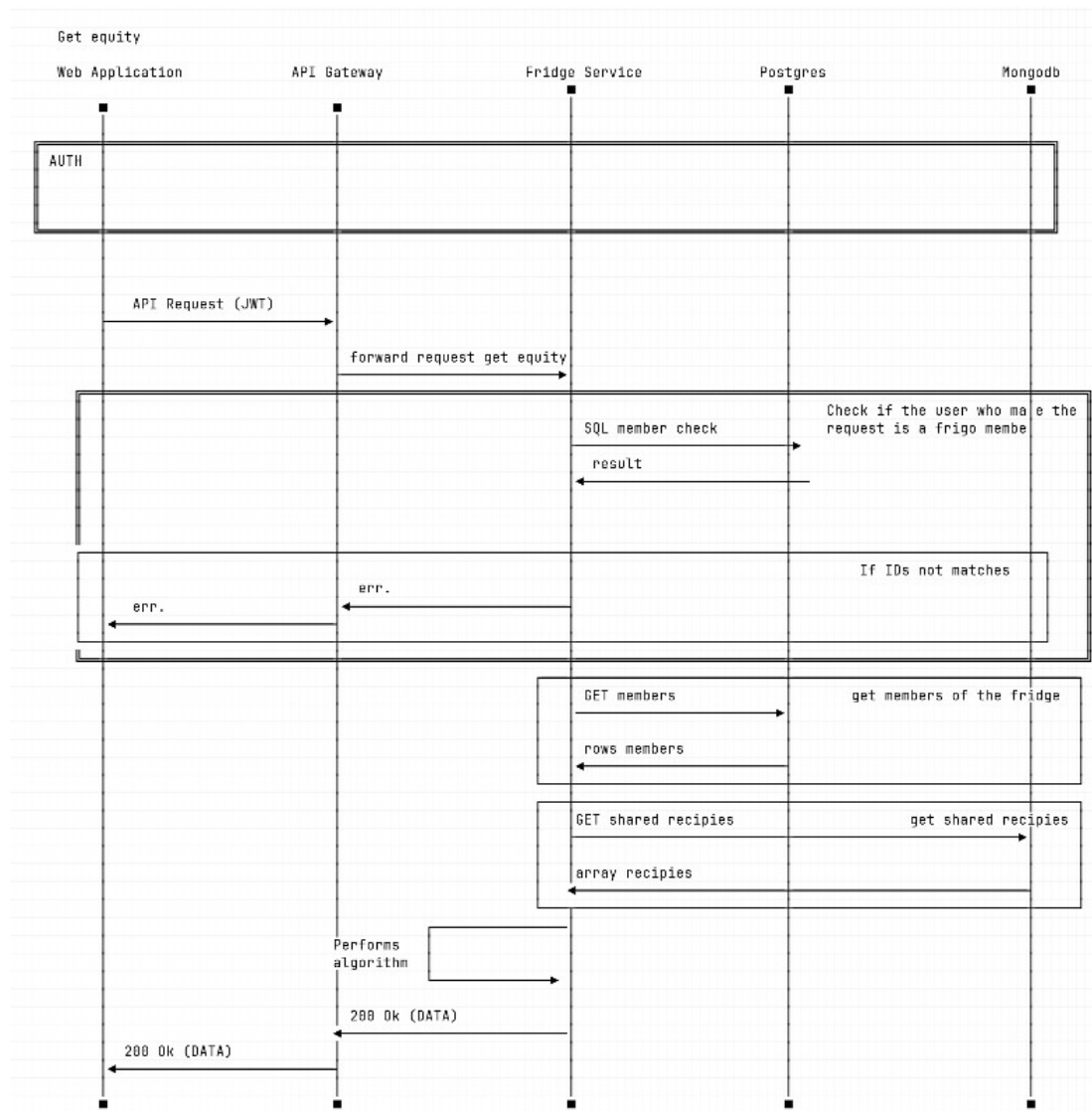


Figura 8.2: Sequence Diagram – Calcolo Distribuzione Equa dei Contributi (UC10)

8.3.3 Rimozione Utente dal Frigorifero

Il diagramma illustra il flusso di rimozione forzata di un membro (UC12), eseguibile esclusivamente dall'utente con ruolo OWNER. Il frontend invia una richiesta DELETE `/api/v1/fridges/:fridgeId/members/:userId` al Gateway, che dopo la verifica del token inietta le credenziali dell'utente richiedente negli header. Il Fridge Service verifica innanzitutto che il richiedente sia OWNER del frigorifero tramite una query su `FRIDGE_MEMBERS`; se il controllo fallisce risponde 403 **Forbidden**. Se il controllo passa, il servizio elimina il record corrispondente all'utente target in `FRIDGE_MEMBERS` e restituisce 200 **OK**. Il membro rimosso, al successivo accesso all'applicazione, non vedrà più il frigorifero nella propria Dashboard poiché la query `GET /fridges` filtra i risultati sulla base dell'appartenenza corrente.

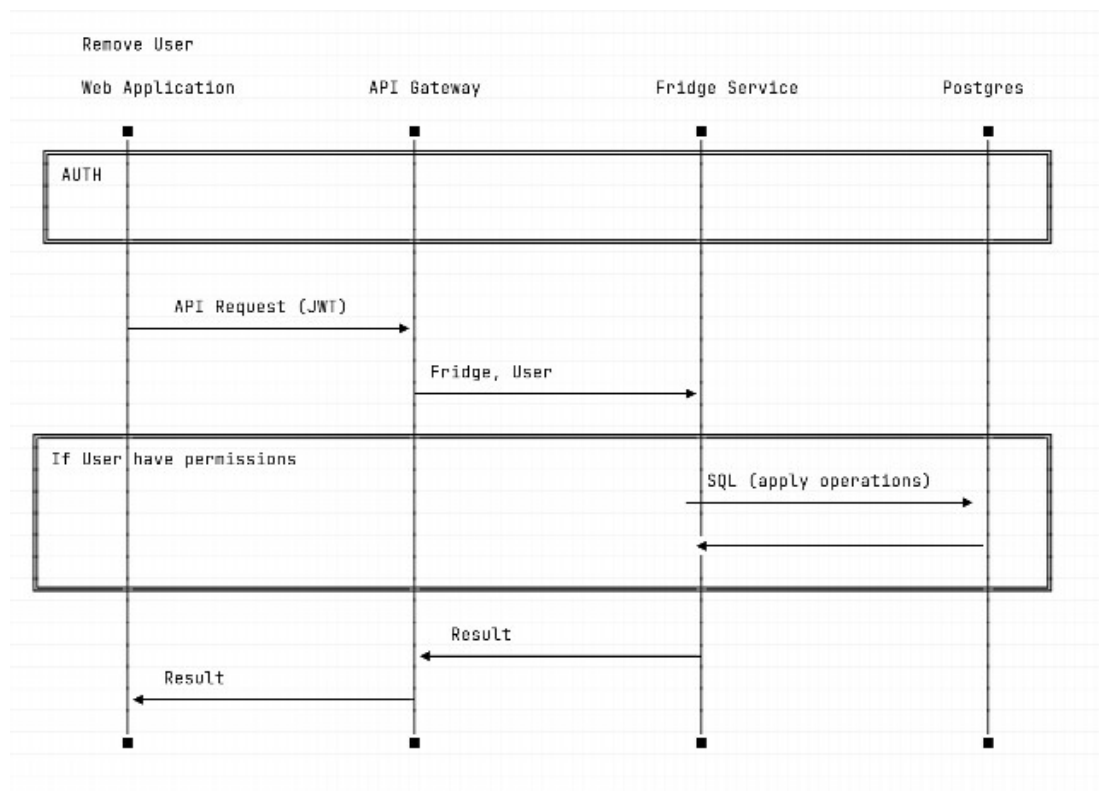


Figura 8.3: Sequence Diagram – Espulsione di un Membro dal Frigorifero (UC12)

8.4 Testing – Iterazione 2

8.4.1 Analisi Dinamica – Fridge Service e Recipe Service

Nell'Iterazione 2 la copertura dei test è stata estesa al **fridge-service** e al **recipe-service**. I test del **fridge-service** coprono i nuovi endpoint relativi alla gestione dei membri (rimozione, uscita) e al calcolo dell'equità. I test del **recipe-service** verificano il corretto inoltro della richiesta all'API OpenAI e la gestione degli errori in caso di risposta non valida.

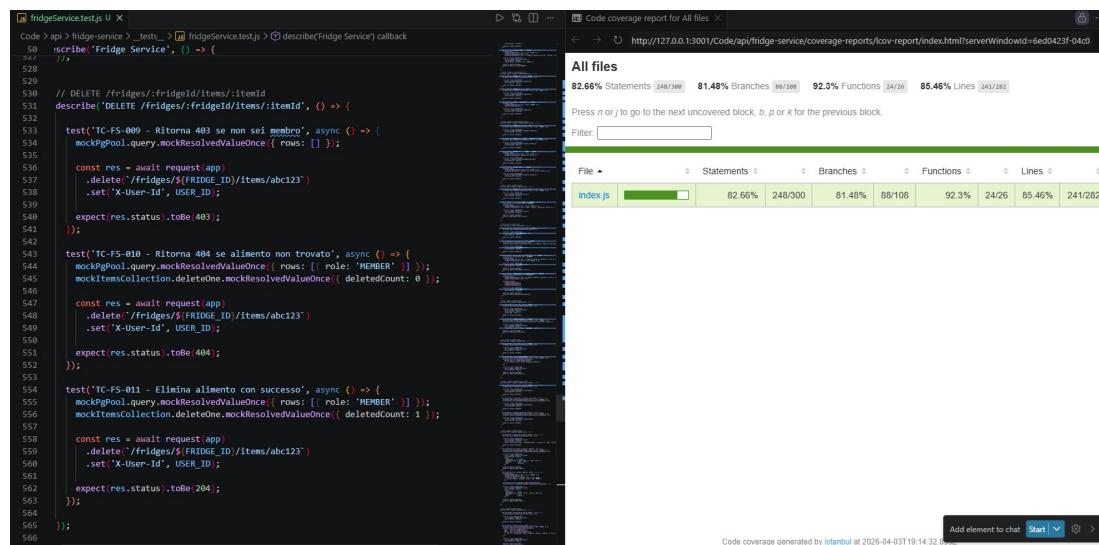


Figura 8.4: Report di Coverage – Fridge Service (Iterazione 2)

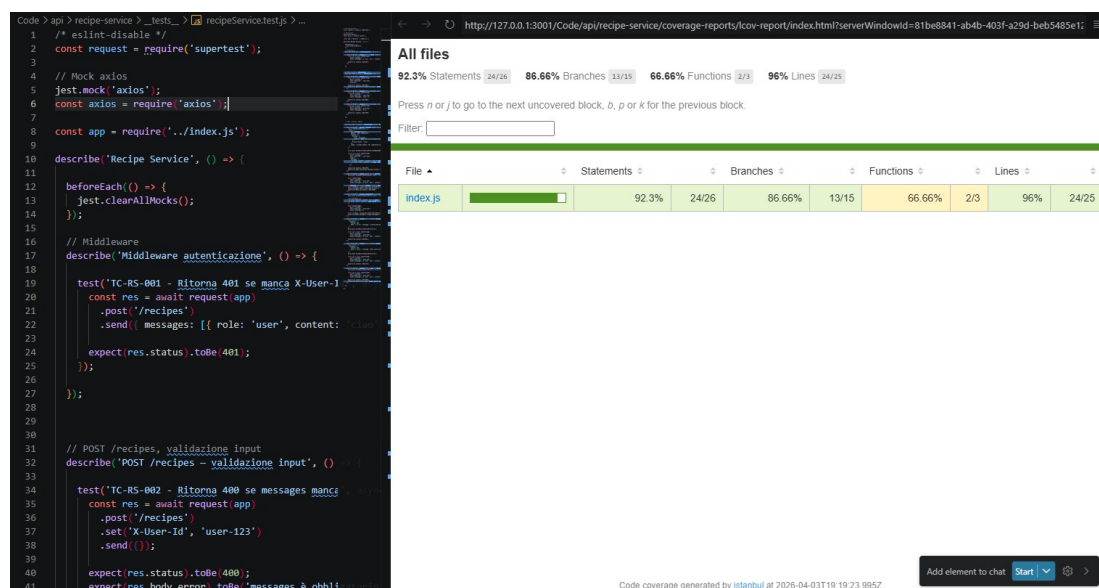


Figura 8.5: Report di Coverage – Recipe Service (Iterazione 2)

8.5 Documentazione API – Iterazione 2

Le tre API introdotte in questa iterazione coprono le funzionalità di equità, generazione ricette e gestione dei membri.

8.5.1 Calcolo Equità e Generazione Ricetta

GET /api/v1/fridges/:fridgeId/equity non accetta parametri nel body e non modifica alcuno stato; è un endpoint di sola lettura che aggrega dati da PostgreSQL e MongoDB in tempo reale. La risposta è un oggetto JSON con chiave `userId` e valore il bilancio del contribuuto, espresso nell'unità monetaria stimata.

POST /api/v1/recipes riceve nel body un array `messages` contenente la lista degli ingredienti selezionati formattata come prompt. Il Recipe Service la inoltra direttamente all'API OpenAI e restituisce al client la risposta dell'LLM senza ulteriori elaborazioni, delegando al frontend il parsing e la visualizzazione delle ricette suggerite.

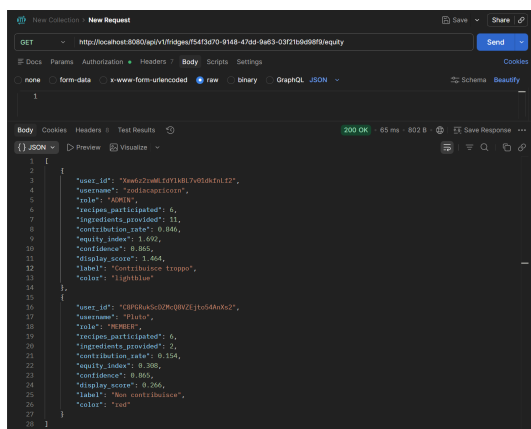


Figura 8.6: GET /api/v1/fridges/:id/equity

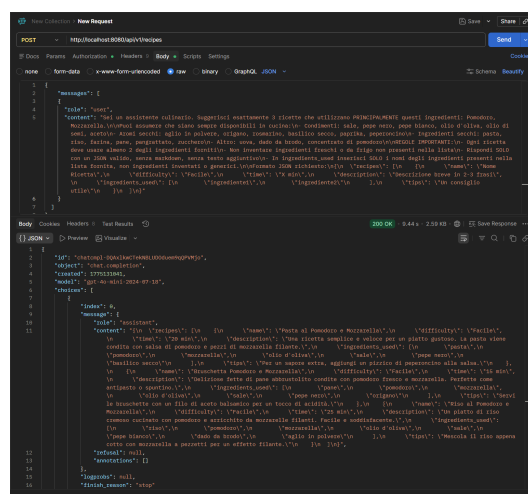


Figura 8.7: POST /api/v1/recipes

8.5.2 Rimozione Membro

DELETE /api/v1/fridges/:fridgeId/members/:userId rimuove il membro identificato da `userId` dal frigorifero. Come illustrato nel sequence diagram corrispondente, l'operazione è riservata all'OWNER: il Fridge Service verifica il ruolo del richiedente prima di procedere e risponde 403 Forbidden se il controllo fallisce. In caso di successo risponde 200 OK con un messaggio di conferma; il membro rimosso perde immediatamente l'accesso all'inventario.

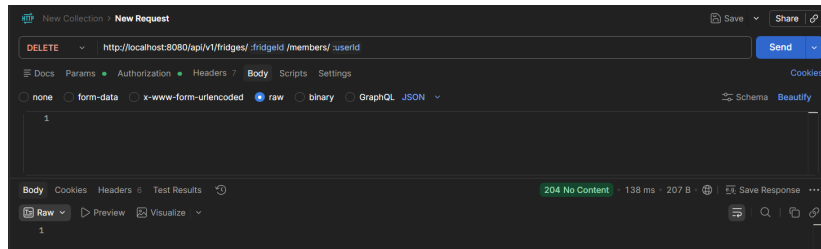


Figura 8.8: DELETE `/api/v1/fridges/:id/members/:userId`

9. Infrastruttura CI/CD

9.1 Continuous Integration e Continuous Delivery

All'interno dell'odierno panorama dell'ingegneria del software, l'implementazione di pratiche di automazione per il rilascio del codice risulta fondamentale per garantire scalabilità, affidabilità e rapidità d'esecuzione. In questo progetto è stata implementata una pipeline di **Continuous Integration e Continuous Delivery (CI/CD)** basata su **GitHub Actions**, specificamente ingegnerizzata per gestire un'architettura a microservizi ospitata all'interno di un monorepo. L'infrastruttura è progettata per automatizzare rigorosamente i processi di validazione, compilazione e pacchettizzazione del software, azzerando l'intervento manuale e riducendo significativamente il margine di errore sistemico.

9.2 Triggers e Filtri di Percorso

Il processo di automazione è configurato per avviarsi in modo continuo a ogni modifica del codice sul ramo principale (**main**), fungendo da barriera di transizione tra lo sviluppo e l'ambiente di produzione. È altresì prevista l'esecuzione manuale (**workflow_dispatch**) per garantire flessibilità operativa in scenari di ripristino o rilasci forzati.

Operando in un contesto monorepo, la pipeline implementa **filtri di percorso** per identificare in modo granulare quali specifici microservizi (es. **gateway**, **fridge-service**, **user-service**) abbiano subito mutazioni. Questo approccio selettivo garantisce un'ottimizzazione delle risorse computazionali, istruendo il sistema a eseguire i successivi processi di validazione e compilazione esclusivamente per i moduli effettivamente modificati, riducendo drasticamente i tempi di esecuzione complessivi (*time-to-market*).

La figura 9.1 mostra la struttura del grafo di esecuzione (*workflow DAG*) nello stato *Queued*, evidenziando come il job **detect-changes** si ramifichi in parallelo verso i job di test e build per ogni servizio modificato. La figura 9.2 mostra lo stesso workflow al termine dell'esecuzione con stato *Success*, confermando il completamento di tutte le fasi.

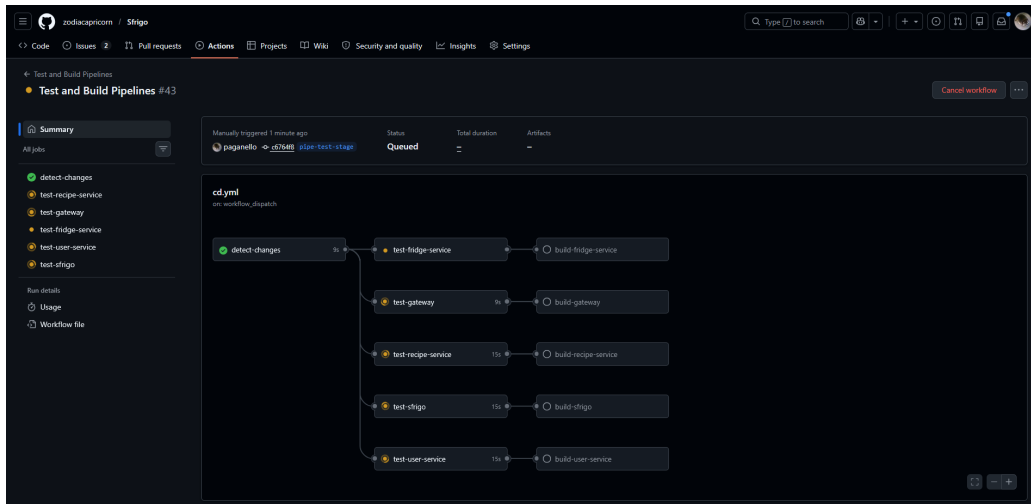


Figura 9.1: Workflow DAG – stato *Queued*

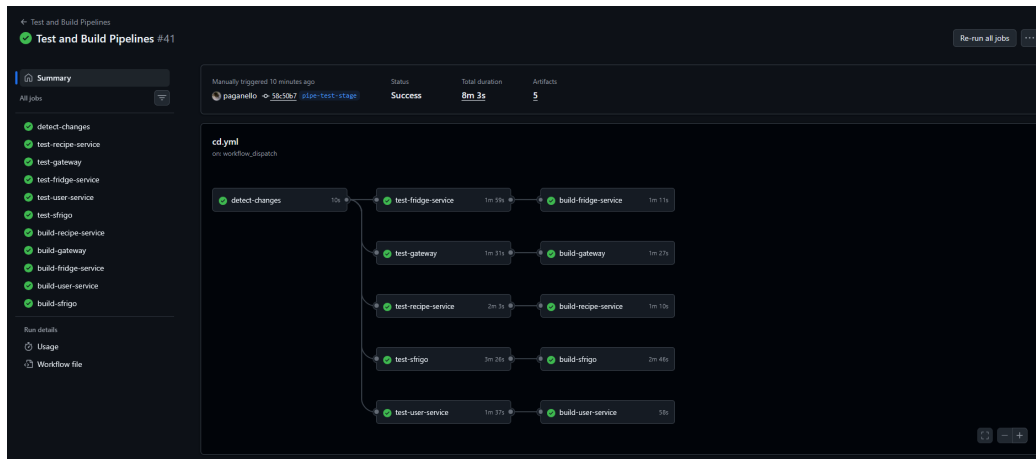


Figura 9.2: Workflow DAG – stato *Success*

9.3 Flusso della Pipeline

Per ogni modulo, il flusso di lavoro alloca dinamicamente **ambienti di esecuzione isolati** in cui avvia le relative fasi di testing: dopo il setup delle dipendenze, il codice subisce:

- un'analisi architetturale, mirata a prevenire dipendenze circolari;
- controlli di formattazione (*linting*);
- test unitari.

Queste fasi fungono da barriera deterministica contro la propagazione di eventuali regressioni. Esclusivamente a fronte di una validazione superata con successo, la pipeline innesca le fasi di *build* e *push*: previa autenticazione al Docker registry di destinazione tramite credenziali (passate via **Secrets**), il codice sorgente

viene compilato in **immagini Docker**, garantendo la totale standardizzazione e idempotenza del software in ambiente produttivo.

Infine, l'artefatto viene inviato al registry e contrassegnato da una **duplice etichettatura**:

- il tag `latest`, per la versione corrente;
- l'hash univoco del commit, per la referenza esatta.

Questa pratica è necessaria per assicurare la tracciabilità assoluta del ciclo di vita applicativo e garantire operazioni di *rollback*.

La figura 9.4 riporta il log di esecuzione del job `test-fridge-service`, mostrando i test Jest con i relativi casi di test superati. La figura 9.3 mostra la pagina dei *Repository Secrets* di GitHub, dove sono configurate le credenziali del Registry Docker e le variabili d'ambiente di Firebase necessarie al build del frontend.

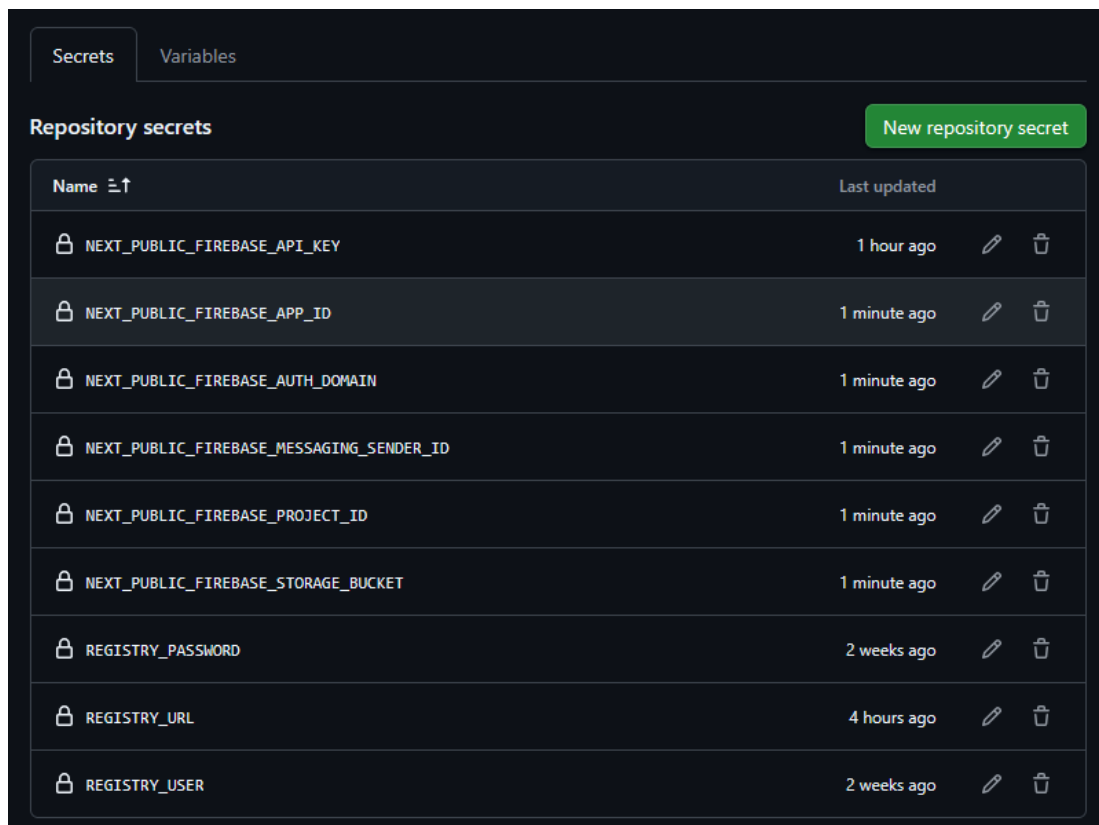


Figura 9.3: Repository Secrets – credenziali CI/CD

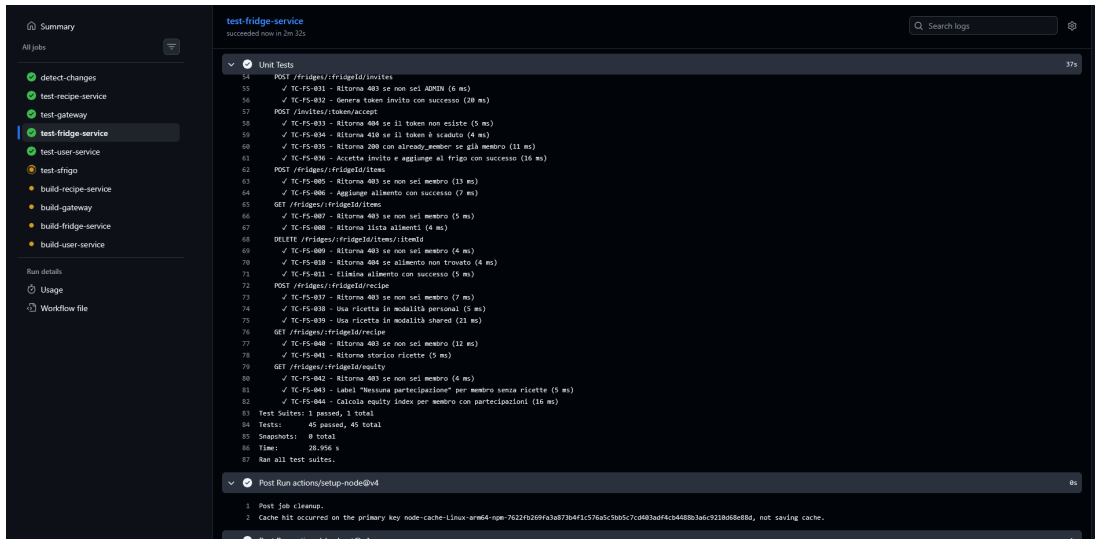


Figura 9.4: Log del job test-fridge-service

9.4 Architettura di Esecuzione

L'infrastruttura computazionale sottostante all'esecuzione delle pipeline CI/CD si fonda su un **cluster Kubernetes** proprietario, implementato e configurato su istanze cloud con architettura ARM fornite da **Oracle Cloud Infrastructure (OCI)**. L'orchestrazione e l'allocazione dinamica dei flussi di lavoro GitHub Actions sono demandate a un **Action Runner Controller (ARC)**, operante in ascolto (*listener*) all'interno del cluster stesso e direttamente interfacciato con il registro delle pipeline.

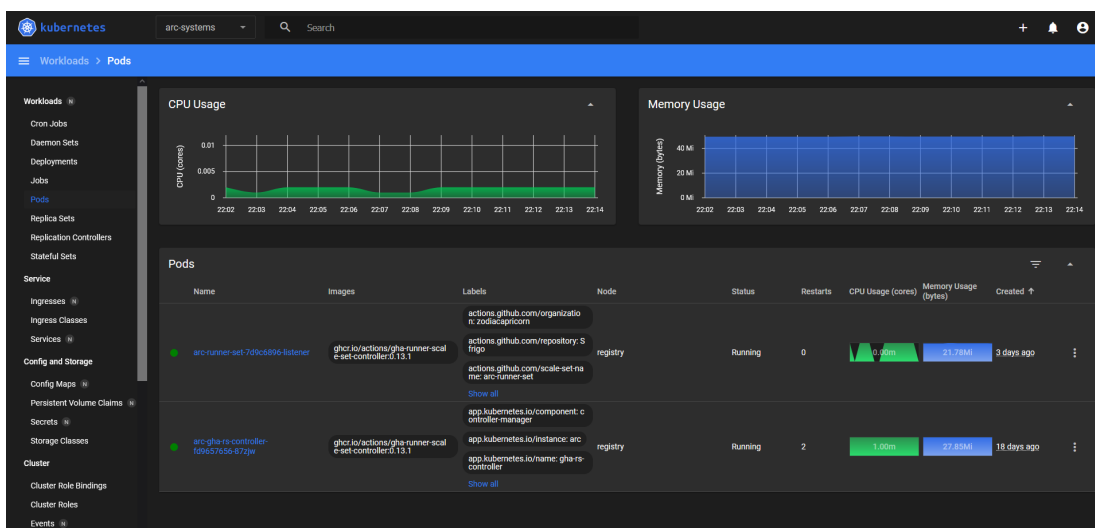


Figura 9.5: Kubernetes – pod con image taggate dopo il deploy automatico

9.5 Architettura di Deployment e Topologia dei Servizi

L'infrastruttura di deployment, ospitata sul cluster Kubernetes precedentemente delineato, si articola in una **topologia a microservizi** orchestrata per garantire un'elevata coesione interna e una netta separazione delle responsabilità (*separation of concerns*). Come si evince dall'analisi dei carichi di lavoro (*workloads*), l'intero ecosistema applicativo è logicamente segregato all'interno di un **namespace dedicato** e si compone di molteplici livelli funzionali.

Il livello di presentazione è gestito da un deployment dedicato al frontend (*frontend-deployment*), affiancato da un **API Gateway** (*gateway*) che funge da mediatore per le logiche di backend. I microservizi preposti alla business logic specifica (identificati dalle immagini *fridge-service*, *recipe-service* e *user-service*) risultano incapsulati all'interno di un aggregato di elaborazione unificato (*api-deployment*), ottimizzando l'allocazione delle risorse interne.

A supporto di questi moduli operativi, lo strato di persistenza dei dati adotta un **approccio poliglotta**, impiegando deployment indipendenti per sistemi di gestione di database sia relazionali (**PostgreSQL**) sia orientati ai documenti (**MongoDB**), al fine di assecondare le eterogenee necessità di modellazione dei dati dei singoli servizi. È rilevante notare come il provisioning di tutte le componenti applicative (ad eccezione dei database standard) avvenga tramite il *pull* dinamico delle immagini dal registro locale del cluster, chiudendo di fatto il ciclo di Continuous Delivery. Infine, l'esposizione dell'intero applicativo verso la rete esterna è delegata e centralizzata tramite una risorsa **Ingress**. Questo strato architetturale agisce come punto di ingresso unificato e *reverse proxy*, occupandosi della risoluzione dei nomi a dominio e del routing del traffico HTTP/HTTPS dall'esterno verso i corrispondenti servizi interni al cluster.

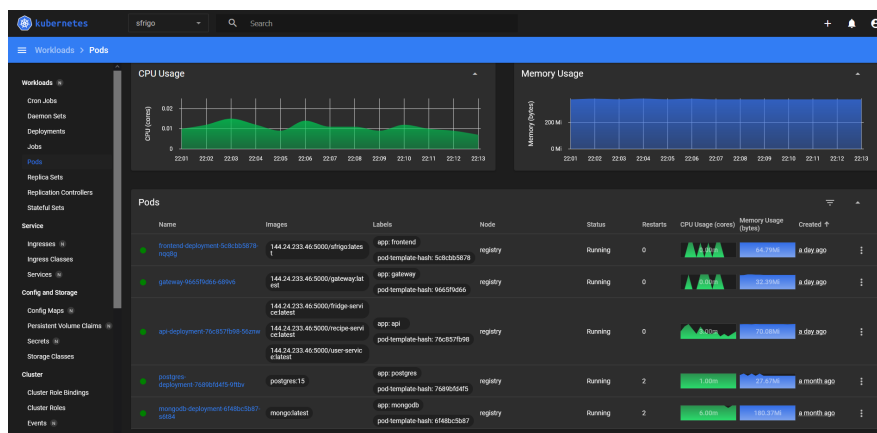
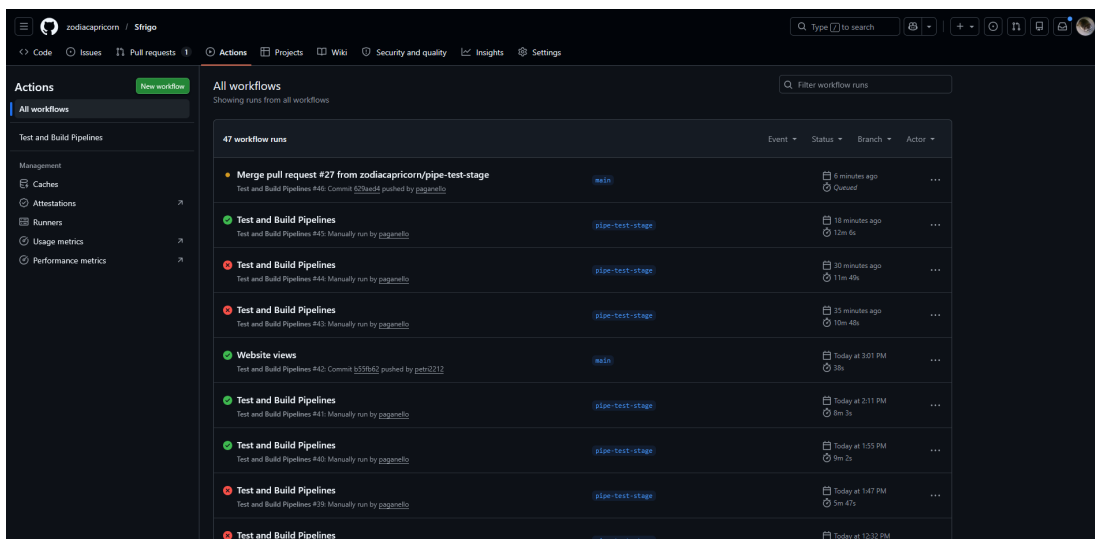


Figura 9.6: Kubernetes – namespace *sfrigo* con pod in esecuzione e metriche CPU/Memory

9.6 Implicazioni Strategiche e Aziendali

L'adozione di questo paradigma metodologico risponde in modo esaustivo alle esigenze aziendali moderne. Strategicamente, la pipeline converte il consolidamento del codice in un rilascio tangibile e sicuro, spostando il controllo di qualità a monte del processo (*shift-left testing*). Questo isola le dipendenze dei vari microservizi, permettendo ai team di sviluppo di rilasciare aggiornamenti in totale autonomia e parallelismo, massimizzando l'efficienza delle risorse, mitigando il rischio di deployment e garantendo un flusso continuo e ininterrotto di valore verso l'utente finale.



The screenshot displays the GitHub Actions interface for the repository 'zodiacapricorn / Sfrigo'. The left sidebar shows the 'Actions' tab with a 'New workflow' button. The main area is titled 'All workflows' and shows a list of workflow runs. The table below represents the data shown in the screenshot.

Event	Status	Branch	Actor
Merge pull request #27 from zodiacapricorn/pipe-test-stage	Completed	main	github
Test and Build Pipelines	Completed	pipe-test-stage	github
Test and Build Pipelines	Completed	pipe-test-stage	github
Test and Build Pipelines	Completed	pipe-test-stage	github
Test and Build Pipelines	Completed	pipe-test-stage	github
Website views	Completed	main	github
Test and Build Pipelines	Completed	pipe-test-stage	github
Test and Build Pipelines	Completed	pipe-test-stage	github
Test and Build Pipelines	Completed	pipe-test-stage	github
Test and Build Pipelines	Completed	pipe-test-stage	github
Test and Build Pipelines	Completed	pipe-test-stage	github

Figura 9.7: GitHub Actions – storico dei workflow run con indicazione degli stati

10. Viste Frontend



Figura 10.1: Homepage del sito web

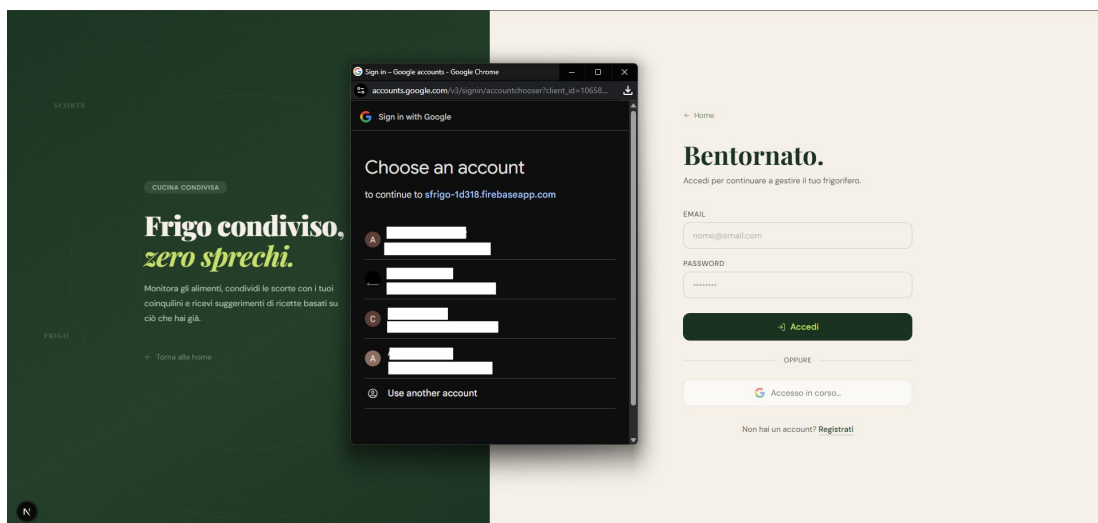


Figura 10.2: Schermata di Login

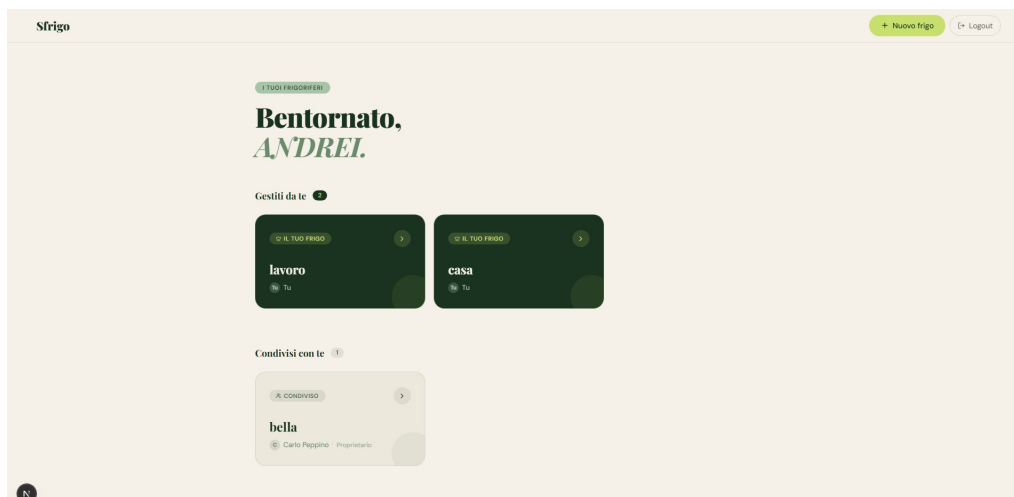


Figura 10.3: Dashboard: i tuoi frigoriferi condivisi

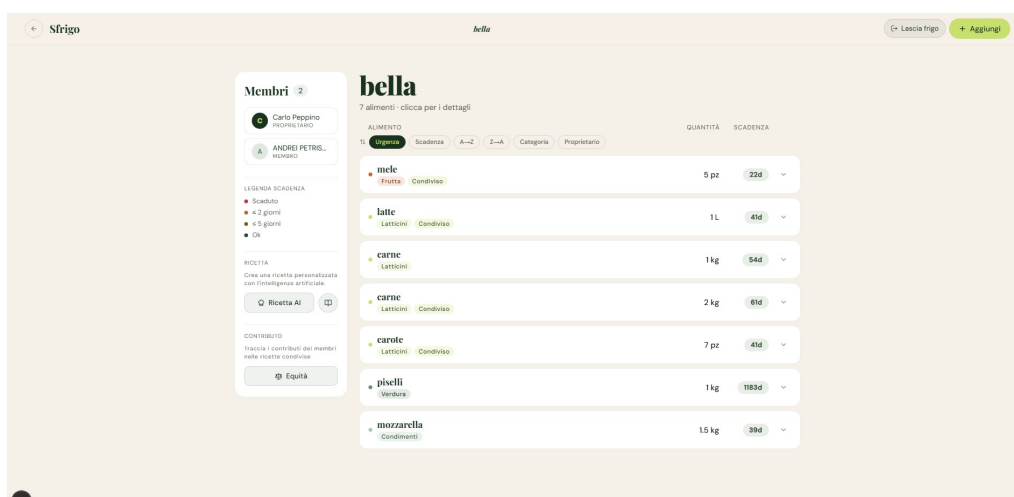


Figura 10.4: Inventario del frigorifero con modalità di ordinamento per urgenza

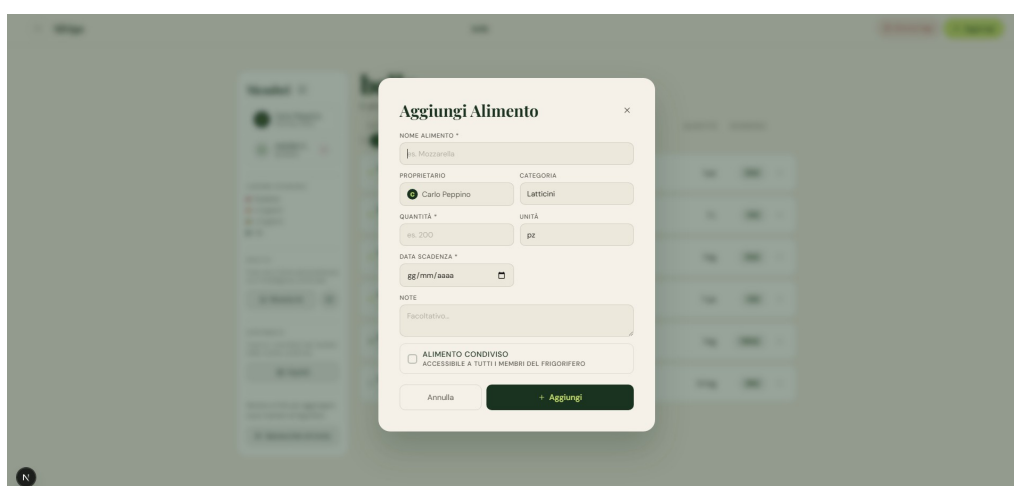


Figura 10.5: Aggiungi Alimento

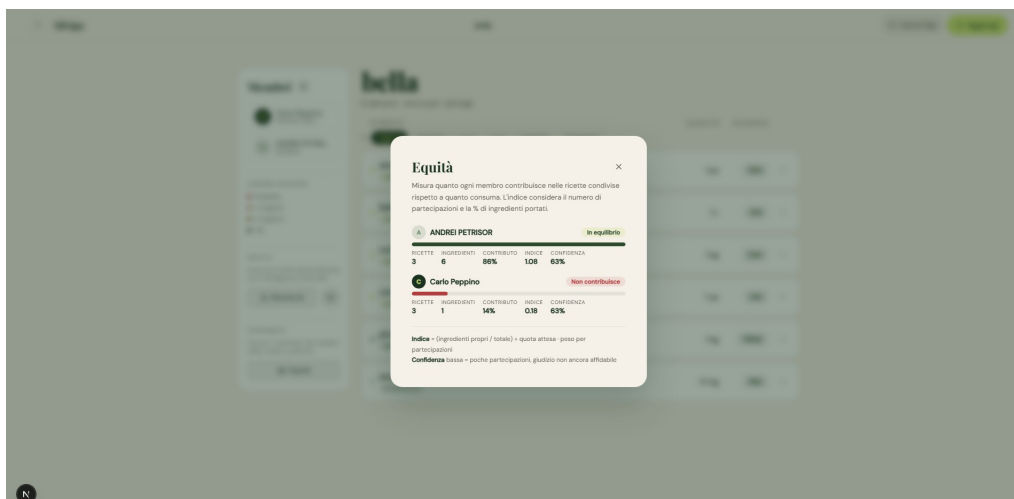


Figura 10.6: Algoritmo di equity

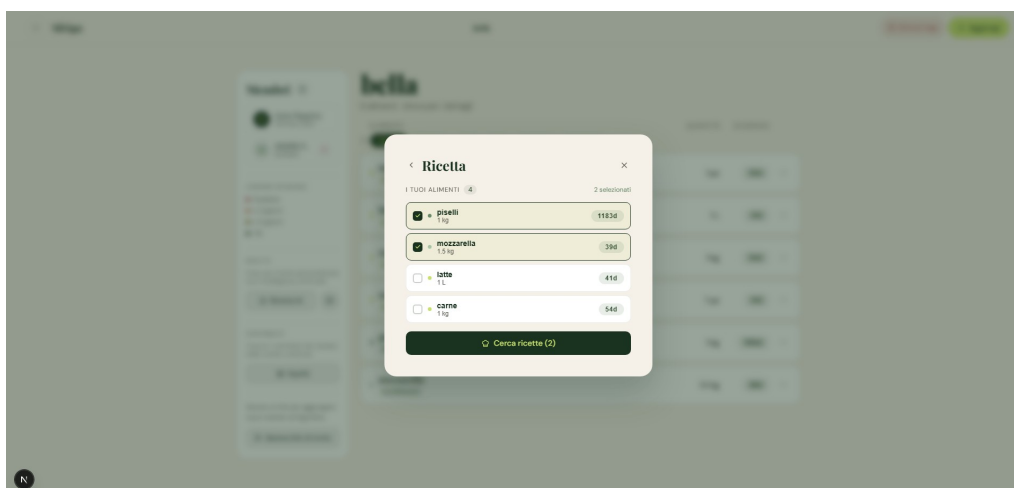


Figura 10.7: Ricetta: visualizzazione e modifica

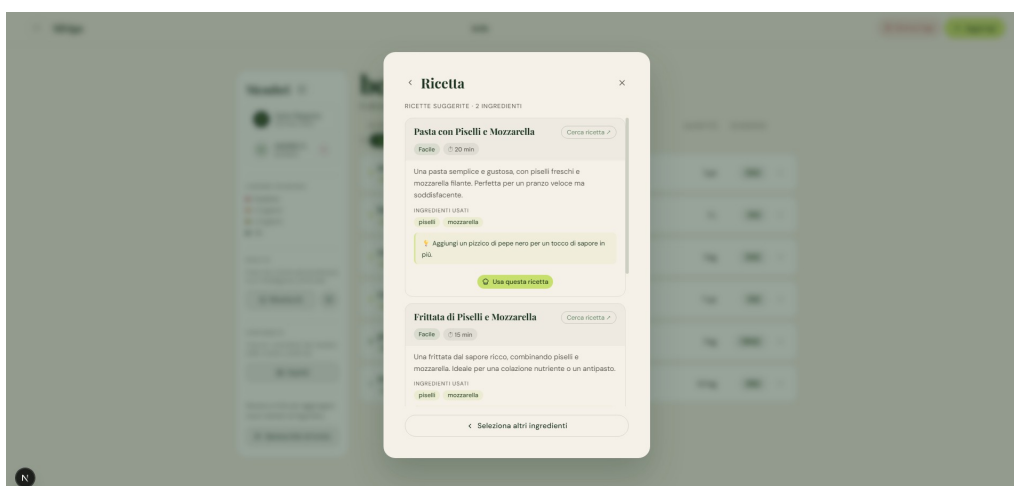


Figura 10.8: Ricette suggerite